

CHAPTER 1

1.1 Introduction

This chapter provides a comprehensive introduction to the project “Trade Pattern Extraction and Financial Market Structure Discovery for Quantitative Trading”. It begins with a general overview of financial time-series analysis and algorithmic trading, followed by a discussion of the present-day landscape of quantitative research. The chapter then outlines the motivation behind this work, identifying key shortcomings in existing methodologies and articulating the importance of a data-driven, automated approach to market pattern discovery. The primary and secondary objectives of the project are formally stated along with the target specifications. The chapter concludes with the project work schedule and an organizational overview of complete report.

1.2 Introduction to the Area of Work

Financial markets produce enormous volumes of time-series data every trading session, encoding rich information about price movements , returns , volatility regimes , and market microstructure. Quantitative trading — the discipline of applying mathematical and statistical models to derive systematic trading signals – has grown exponentially over the past two decades , driven by the availability of high-frequency data , increased computational power , and advances in machine learning.

Time-series pattern mining is a foundational task within quantitative finance. A “pattern” in this context refers to a recurring subsequence or structural configuration in price data that precedes a statistically significant market move. Examples include classical technical formations such as breakouts , pullbacks , consolidations , and mean-reversion setups. Historically , these patterns were identified manually by traders using heuristic rules , candlestick charts , and fixed technical indicators such as Moving Average Convergence Divergence (MACD) , Relative Strength Index (RSI) and Bollinger Bands.

Machine learning and data mining have begun transforming this field . Feature engineering libraries such as TSFresh enable automatic extraction of hundreds of descriptive statistics from raw time-series windows , capturing momentum , entropy , autocorrelation structure , and non-linear characteristics that manual indicators cannot. Advanced similarity measures such as the Matrix Profile (a technique for identifying all pairs time-series subsequence similarities using the z-normalized Euclidean distance) and Dynamic Time Warping (DTW), which compares sequences allowing for temporal elasticity, have opened new avenues for robust , automated pattern discovery. These approaches go beyond fixed- rule matching , enabling the detection of structurally . These approaches go beyond fixed-rule matching , enabling the detection of structurally equivalent market behaviour across varying time scales and market regimes.

This project sits at the intersection of quantitative finance, time-series analysis, machine learning , data mining , and scientific computing. It combines feature engineering (TSFresh),

sequence-similarity mining (Matrix Profile via STUMPY), and elastic distance measures (DTW) into a unified , modular pipeline designed to discover , expand , validate and visualize recurring trade setups directly from historical financial data.

1.3 Present-Day Scenario

The contemporary algorithmic trading landscape is dominated by systematic strategies that rely on quantitative signals derived from price , volume and alternative data. Hedge funds, proprietary trading firms, and institutional asset managers globally invest significant resources in developing proprietary pattern recognition and signal extraction pipelines. According to industry estimates , algorithmic and high-frequency trading now accounts for over 60-70% of daily equity market volume in developed markets.

Despite this growth , a majority of retail and mid-tier quant practitioners still depends on manually crafted technical indicators and shallow rule-based systems. These approaches suffer from overfitting to specific market conditions and fail to generalize across market regimes, asset classes , or time horizons. There is growing recognition in the academic and industry literature that automated , data-driven framework for pattern extraction offer superior robustness and adaptability.

Open-source tools such as STUMPY for matrix profile computations, TSFresh for automated feature engineering, and DTW libraries have matured considerably and are now production-grade . Interactive visualization platforms like bokeh provide rich, browser-based tools for inspecting discovered patterns. The convergence of these technologies creates a timely opportunity to build an end-to-end quantitative pattern mining framework that is both scientifically rigorous and practically usable by traders and researchers.

This project , conducted as an internship engagement with Xcitium India Services Private Ltd (NuFinTech Department), directly address this industrial need by building such a framework using python-based open-source tooling.

1.4 Motivation

1.4.1 Shortcomings in Previous Work

Traditional technical analysis and indicator-based trading exhibit several well-documented limitations that motivate the development of systematic alternative:

- Poor generalization across market regimes: Fixed-parameter indicator such as RSI(14) OR MACD(12,26,9) are tuned to specific historical periods. Their performance degrades significantly when market volatility regimes shift, making them unreliable in live trading.

- Inability to detect time warped patterns : Markets can produce structurally equivalent patterns at different speeds — a breakout that takes three bars in a fast market may take ten bars in a slow market. Classical Euclidean distance-based matching fails to identify such correspondences. DTW address this by allowing non-linear temporal alignment.
- Manual , error-prone feature engineering: Constructing a comprehensive feature set from raw OHLCV data requires domain expertise , is labour-intensive ,and is prone to look-ahead bias if temporal boundaries are not strictly enforced .TSFresh automates this rigorously.
- Lack of statistical validation: Many retail pattern-based strategies are deployed without formal hypothesis testing , leading to unquantified out-of-sample risk. A rigorous validation framework is absent from most existing tools.
- Scalability limitations: Manual analysis cannot scale to thousands of instruments , multiple time frames, and years of tick data simultaneously. Automated pipelines are essential for institutional-grade research.

1.4.2 Importance of the Work in the Present Context

The ability to automatically discover and validate recurring market pattern has direct practical value. Systematic strategies grounded in data-driven patterns are more robust, reproducible , and auditable than discretionary trading approaches. In a regulatory environment that increasingly demands explainability and model documentations (e.g. MiFID II in Europe), automated frameworks that produce structured summaries — offer a significant advantage. Additionally , the democratization of such tools enables smaller research teams and individual quant researchers to compete on analytical capability with larger institutions.

1.4.3 Uniqueness of Methodology

This framework’s methodology is distinguished from prior work in several key ways:

- Integration of three complementary paradigms : TSFresh feature engineering, Matrix Profile pattern mining and DTW-based expansion are combined in a single, coherent pipeline — a combination not seen in existing open-source tools.
- Strict temporal discipline : All feature extraction and pattern discovery operations respect the temporal ordering of data , with features computed exclusively from past observations to eliminate look-ahead bias.
- Dual pattern expansion: Both Euclidean similarity (For fixed-horizon setups) and DTW (for time-warped setups) are supported, giving practitioners flexibility to match Different strategy types
- Interactive visualization via Bokeh : Discovered patterns are rendered with overlay Capability , enabling qualitative inspection alongside quantitative statistics — bridging machine output with trader intuition.

- Modular , extensible design: Each pipeline stage is independently configurable, supporting future extension such as clustering , reinforcement learning-based strategy evaluation, or real-time data feeds.

1.4.4 Significance of the End Result

The framework's end result — validated pattern libraries, ML-ready feature matrices , and interactive dashboards — directly feed into downstream strategy development workflows. Traders can use discovered patterns as entry filters, risk managers can use statistical metrics to assess pattern reliability , and machine learning engineers can use the feature matrices to train classification or regression models on labelled trade setups . The frameworks thus serves as a foundational research infrastructure for quantitative trading teams.

1.5 Objectives

1.5.1 Main Work Objective

The primary objective of this project is to design, implement, and validate an end-to-end automated pipeline for trade pattern extraction and financial market structure discovery from historical time-series data. The pipeline must be capable of:

- Automatically segmenting financial time series into trade-context windows using a sliding-window approach
- Performing large-scale , bias-free feature engineering using TSFresh to produce ML-ready representations of each trade window.
- Discovering recurring market patterns using Matrix Profile-based subsequence matching via the STUMPY library.
- Expanding discovered patterns using Euclidean similarity and Dynamic Time Warping to capture both fixed-horizon and time-warped setups.
- Statistically validation discovered patterns using appropriate hypothesis testing and performance metrics.
- Providing interactive, layered visualization of patterns and their instances using the Bokeh plotting library.

1.5.2 Secondary Objectives

The secondary objectives of the project are :

- To build a modular and extensible codebase that can be adapted for different asset classes , time frames , and financial instruments beyond the initial equity dataset.
- To document the framework comprehensively, enabling reuse by other quantitative researchers and data scientists in the fintech domain
- To explore the relative strengths and weaknesses of Euclidean similarity versus DTW-based pattern expansion in terms of pattern coverage and statistical robustness.
- To create a reusable feature engineering preset system (minimal , efficient, comprehensive) that allows practitioners to balance computational cost and feature richness.
- To assess the framework's scalability to larger datasets and higher-frequency data, identifying computational bottlenecks and proposing optimization paths.

1.6 Target Specifications

The framework is designed to meet the following technical and functional specifications:

- Language & Environment : Python 3.10+ running on a 64-bit system with a minimum of 8 GB RAM (16 GB recommended) , with a minimum of 10 GB free storage. GPU support is optional and not required.
- Key Libraries: NumPy 2.0.2 , Pandas 2.2.2 , Matplotlib 3.10.0 , Bokeh 3.7.3 , Pathos 0.3.4, STUMPY 1.13.9 , TSFresh 0.21.1 , and standard Scikit-learn utilities.
- Input Data : OHLCV (Open, High , Low , Close , Volume) Financial time series from any standard data source (e.g. Yahoo Finance , Quandl m or CSV-formatted exchange data).
- Output : Tabular trade-window dataset , ML-ready TSFresh feature matrices (hundreds of features per window) , discovered pattern objects with statistical summaries , and interactive Bokeh dashboard in HTML format.
- Temporal Integrity : All features must be computed strictly from past data; the framework enforces no look-ahead bias at any pipeline stage.
- Visualization : Pattern overlays , instance clustering plots, and statistical validation summaries must be rendered as interactive HTML artifacts using Bokeh , accessible without requiring a dedicated server.

The importance of these specifications lies in their alignment with production -grade quantitative research standards. Temporal integrity is non-negotiable for any strategy that will be deployed in live trading. The output format is chosen to ensure seamless integration with downstream ML workflows and risk management systems. The hardware requirements are set to ensure broad accessibility while supporting the computational demands of matrix profile computation and large-scale TSFresh extraction.

1.7 Project Work Schedule

The project was executed over a four-month (16 Weeks) period from November 2025 to February 2026, structured as follows:

Month	Activities
November 2025	Literature survey on quantitative trading and pattern mining; dataset collection and preprocessing; Python environment setup and dependency configuration; review of TSFresh, STUMPY, and DTW documentation.
December 2025	Implementation of sliding-window trade context extraction system; integration of TSFresh automated feature engineering; initial feature validation and bias checks; unit testing of the data pipeline.
January 2026	Matrix Profile-based pattern discovery module development using STUMPY; implementation of Euclidean similarity-based pattern expansion; implementation of DTW-based pattern expansion; intermediate testing and parameter tuning.
February 2026	Development of the interactive Bokeh visualization module; implementation of statistical validation framework (metrics computation, hypothesis testing); system integration testing; preparation and submission of project report and final evaluation.

Table 1.1: Project Work Schedule

1.8 Organization of the Project Report

The remainder of this report is organized into five chapters, each addressing a distinct aspect of the project:

Chapter 1 — Introduction: This chapter provides a comprehensive introduction to the project "Trade Pattern Extraction and Financial Market Structure Discovery for Quantitative Trading". It begins with a general overview of financial time-series analysis and algorithmic trading, followed by a discussion of the present-day landscape of quantitative research. The chapter then outlines the motivation behind this work, identifying key shortcomings in existing methodologies — including poor generalization across market regimes, inability to detect time-warped patterns, and the absence of rigorous statistical validation — and articulates the importance of a data-driven, automated approach to market pattern discovery. The primary and secondary objectives of the project are formally stated alongside the target technical specifications governing the framework's design. The chapter concludes with the project work

schedule spanning the four-month execution period and an organizational overview of the complete report.

Chapter 2 — Background Theory and Literature Review: This chapter provides a detailed survey of the foundational concepts and prior work relevant to this project. It covers time-series analysis in financial markets, technical analysis limitations, existing pattern mining approaches, the theoretical basis of Matrix Profile and Dynamic Time Warping, TSFresh and automated feature engineering, and a critical review of key reference papers. The chapter concludes with a synthesis of gaps in the literature that motivate the proposed framework.

Chapter 3 — Methodology: This chapter presents the complete architecture and detailed design of the Trade Pattern Extraction Framework. Each of the five pipeline stages is described in depth: sliding-window extraction, TSFresh feature engineering, Matrix Profile-based discovery, dual pattern expansion (Euclidean and DTW), and statistical validation with interactive visualization. The chapter includes system block diagrams, module specifications, assumptions, and justifications for all design decisions, along with documentation of all tools and libraries employed.

Chapter 4 — Result Analysis: This chapter presents and analyses the experimental results obtained from applying the framework to historical financial data. Results are presented in both graphical and tabular forms, including discovered pattern visualizations, feature importance analyses, DTW versus Euclidean comparison studies, and statistical validation metrics. Deviations from expected behaviour are discussed and justified. The chapter also assesses the environmental and societal implications of automated quantitative trading frameworks.

Chapter 5 — Conclusion and Future Scope: This chapter summarizes the work, restates the problem objectives, and evaluates how well the framework achieved its goals. General conclusions are drawn from the experimental findings, and the broader significance of the results is discussed. At least three concrete directions for future research and development are proposed, covering extensions such as real-time streaming data support, reinforcement learning-based strategy evaluation, and multi-asset generalization.

CHAPTER 2

BACKGROUND THEORY & LITERATURE REVIEW

2.1 Introduction

This chapter presents a thorough review of the theoretical foundations and prior research relevant to the Trade Pattern Extraction and Financial Market Structure Discovery Framework. The chapter begins with a focused discussion of the project's subject area, specifically the problem of recurring pattern discovery in financial time-series data for quantitative trading applications. This is followed by a survey of the present state of the field, covering recent developments in automated feature engineering, time-series similarity measures, and market pattern mining. The background theory section provides formal introductions to the key algorithmic components employed in this work: TSFresh, the Matrix Profile, and Dynamic Time Warping (DTW). A structured literature survey of eight seminal and recent papers is then present in tabular and narrative form. The chapter concludes with a synthesis of the key gaps in existing approaches, the theoretical justifications for the methodology adopted, and conclusions drawn from the survey.

2.2 Introduction to the Project Title (Specific Discussion)

The title of this project — “Trade Pattern Extraction and Financial Market Structure Discovery Framework for Quantitative Trading” — encapsulates three interrelated research challenges that sit at the intersection of data mining, financial market microstructure, and applied machine learning.

The First challenge is trade pattern extraction: the task of automatically identifying, from raw historical price data, meaningful subsequence or windows of market activity that represent recognizable structural setups. Unlike classical technical analysis, which defines patterns through human-prescribed rules (e.g., a “head and shoulders” formation or a “double bottom”), an automated extraction approach treats the problem as one of subsequence characterization and feature representation. A sliding-windows decomposition of the time series converts each temporal segment into a row of a tabular dataset, and TSFresh automates the derivation of hundreds of statistical descriptions — including measures of trend, entropy, autocorrelation, and nonlinearity — from each window.

The second challenge is market structure discovery: given a larger set of characterized trade windows, the task is to identify which windows recur across history with structural similarity. This is fundamentally a pattern mining problem. The Matrix Profile — a data structure that computes the z-normalized Euclidean distance between every pair of time-series subsequence of a given length — provides an efficient and parameter-light mechanism for finding the most frequent and most similar recurring structures, known as motifs.

The third challenge is pattern generalization: once a core recurring pattern is identified, the goal is to expand its coverage to also include time-warped manifestations — instances where the same structural shape occurs faster or slower than the reference. Dynamic Time Warping provides an elastic distance measure that accommodates this non-linear temporal distortion, substantially increasing the practical yield of discovered pattern.

Together these three components address the full pipeline from raw market data to statistically validated , visually inspectable trade structure — a capability directly applicable to systematic strategy development in quantitative finance.

2.3 Present State and Recent Developments in the Work Area

Quantitative trading research has experienced accelerating growth in the application of machine learning and data-driven methodologies over past decade . Several notable recent developments are directly relevant to this project.

2.3.1 Automated Machine Learning for Financial Feature Engineering

Traditional quantitative strategies rely on a small set of manually selected indicators derived from prices and volume data. Recent work has demonstrated that automated feature extraction — particularly libraries such as TSFresh (Christ el al., 2018) — can systematically generate comprehensive feature sets from raw time-series and outperform hand-crafted indicators for classification and regression tasks. TSFresh implements over 63 feature extraction methods producing up to 794 features per time-series windows, covering statistical moments , entropy , autocorrelation structures , and nonlinear dynamical properties. Its relevance filter (based on the Benjamini-Yekutieli procedure for false discovery rate control) further reduces the feature set to statistically significant predictors. The framework has been applied in industrial process monitoring, anomaly detection, and , more recently, financial time-series classification.

2.3.2 Matrix Profile for Time-Series Motif Discovery

The Matrix Profile, introduce by Yeh et al. (2016) and subsequently extended in a series of papers from the UCR Time Series Center, represents a significant methodological breakthrough for unsupervised pattern mining. It computes, for every subsequence of length m in a time series, the distance to its nearest non-overlapping neighbour , and stores these distances in a compact data structure. Motifs (high recurring patterns) correspond to low-valued entries , while discord (anomalies) correspond to high-valued entries. The STUMPY library (Law, 2019) provides a highly optimized, parallelized Python implementation of Matrix Profile computation, including GPU support, making it practical for datasets with millions of data points . Recent applications in finance include volatility regime detection , high-frequency trade pattern identification, and anomaly-based risk flagging.

2.3.3 Dynamic Time Warping in Financial Applications

Dynamic Time Warping, originally developed for speech recognition (Sakoe and Chiba, 1978), has seen extensive adoption in financial time-series analysis due to its ability to compare sequences that exhibit temporal misalignment or varying speeds. Unlike Euclidean distance, which requires exact point-to-point alignment, DTW finds the optimal non-linear alignment path between two sequences by solving a dynamic programming problem. Studies by Banko et al. (2011) and subsequent work have shown that DTW-based similarity measures significantly outperform Euclidean distance for stock return clustering, candlestick pattern matching, and market regime classification. The key limitation of DTW — its $O(nm)$ computational complexity — is mitigated in practice through constraint-based pruning (Sakoe-Chiba band) and early termination strategies.

2.3.4 Deep Learning Approaches and Their Limitations

While deep learning models such as Long Short-Term Memory (LSTM) networks, Temporal Convolutional Networks (TCN), and Transformer-based architectures have demonstrated strong performance on financial forecasting benchmarks, they introduce significant challenges for practical quantitative research. These include the need for very large labelled datasets, computational expense, poor interpretability, and high susceptibility to non-stationarity. For pattern mining specifically — where the goal is explainable, reusable structural discovery rather than black-box prediction — classical data mining methods grounded in explicit distance measures offer superior transparency and auditability. This project deliberately prioritizes interpretable, statistically grounded methods.

2.3.5 Interactive Visualization for Quantitative Research

The emergence of rich, browser-based visualization platforms such as Bokeh, Plotly, and D3.js has significantly improved the ability of researchers and traders to qualitatively validate algorithmically discovered patterns. Modern quantitative workflows increasingly combine statistical output with interactive overlays, hover-based inspection, and zooming tools to bridge the gap between automated analysis and trader intuition. Bokeh, used in this project, supports server-free standalone HTML output — critical for research environments without dedicated web infrastructure.

2.4 Brief Background Theory

2.4.1 Financial Time Series and the Sliding Window Model

A financial time series is a sequence of observations $X = \{x_1, x_2, \dots, x_N\}$ indexed by discrete time steps, where each observation may represent a closing price, log return, realized volatility, or derived indicator. A sliding window of length m decomposes X into a set of overlapping subsequences $S_i = \{x_i, x_{i+1}, \dots, x_{i+m-1}\}$ for $i = 1, \dots, N-m+1$. Each subsequence S_i represents a potential trade context — a segment of market history of fixed duration. The assumption underlying this framework is that structural regularities in market behaviour manifest as

recurring subsequences that are similar under an appropriate distance measure. Feature engineering transforms each S_i into a feature vector f_i in $\mathbb{R}^d$, enabling tabular machine learning and pattern mining on the resulting dataset.

2.4.2 TSFresh: Automated Time-Series Feature Extraction

TSFresh (Time Series Feature Extraction on basis of Scalable Hypothesis tests) is an open-source Python library that automatically computes a large set of time-series characteristics from a collection of time-series windows. For each window, TSFresh extracts features from several categories including descriptive statistics (mean, median, variance, kurtosis, skewness), momentum and trend measures (linear trend, absolute energy), autocorrelation-based features (ACF, PACF coefficients at specified lags), entropy measures (approximate entropy, sample entropy, permutation entropy), frequency-domain features (via FFT coefficients), and nonlinear dynamical systems features (Hurst exponent, number of crossing points). The library supports three preset configurations: minimal (fast, small feature set), efficient (default, balanced), and comprehensive (all implemented features). Feature generation is strictly window-internal — no data from outside the window is accessed — ensuring temporal integrity. This directly addresses the look-ahead bias problem that plagues many hand-crafted indicator implementations.

2.4.3 Matrix Profile: Theory and Computation

Given a time series T of length n , and a subsequence length m , the Matrix Profile MP is a vector of length $n-m+1$ where $MP[i]$ is the z -normalized Euclidean distance between $T_{i:i+m}$ (the subsequence starting at position i) and its nearest neighbor in T , excluding trivial matches (subsequences within a distance of $m/2$ from i). Formally, let $Z_i = (T_{i:i+m} - \text{mean}(T_{i:i+m})) / \text{std}(T_{i:i+m})$ be the z -normalized subsequence at position i . Then $MP[i] = \min_{\{j-i > m/2\}} \|Z_i - Z_j\|_2$. An associated Matrix Profile Index MPI stores the position of each nearest neighbor. Motifs are pairs $(i, MPI[i])$ with the lowest values in MP , representing the most similar recurring pairs. The STUMPY library computes the Matrix Profile using the STOMP algorithm (Zhu et al., 2016), which achieves $O(n^2)$ time complexity with practical optimizations making it tractable for data series of hundreds of thousands of points.

2.4.4 Dynamic Time Warping: Formal Definition

Given two sequences $Q = \{q_1, \dots, q_m\}$ and $C = \{c_1, \dots, c_n\}$ of possibly different lengths, DTW finds the optimal alignment path $W = \{w_1, w_2, \dots, w_K\}$ where each element $w_k = (i_k, j_k)$ represents a mapping between $Q[i_k]$ and $C[j_k]$. The path must satisfy: boundary conditions ($w_1 = (1, 1)$ and $w_K = (m, n)$), monotonicity ($i_k \leq i_{k+1}$ and $j_k \leq j_{k+1}$), and continuity ($i_{k+1} - i_k \leq 1$ and $j_{k+1} - j_k \leq 1$). The DTW distance is defined as $DTW(Q, C) = \min_W \sqrt{\sum_k d(Q[i_k], C[j_k])^2 / K}$, where $d(q, c) = |q - c|$. The solution is obtained via dynamic programming in $O(mn)$ time. For the pattern expansion step in this project, DTW enables matching of variable-length market subsequences against a fixed reference pattern, accommodating the natural temporal elasticity of market behavior. The `length_range` parameter controls the allowed

ratio between candidate and reference lengths, enabling targeted search over time-scaled versions of the core pattern.

2.4.5 Statistical Validation in Pattern Mining

A discovered pattern is considered statistically valid if its recurrence frequency, cross-instance structural consistency, and out-of-sample predictive behaviour are significantly different from what would be expected under a null model of random walk dynamics. Key validation metrics include: occurrence count (number of discovered instances, used to assess pattern frequency), mean pairwise distance (average z-normalized Euclidean distance across all instance pairs, measuring structural consistency), distance variance (measures stability of similarity across instances), DTW alignment length variability (coefficient of variation of instance lengths, measuring temporal elasticity), and the Motif Significance Score (a z-score of the observed motif distance relative to the distribution of random subsequence distances at the same window length). Only patterns meeting minimum thresholds on occurrence count and consistency metrics are retained for downstream analysis.

2.5 Literature Survey

The following table summarizes eight key publications directly relevant to this project, spanning foundational algorithmic contributions and recent applied finance research. Each entry identifies the methodology, key findings, and limitations that inform the design of the proposed framework.

Ref.	Authors & Year	Title / Contribution	Key Findings	Limitations / Gaps
[1]	Yeh et al. (2016)	Matrix Profile I: All Pairs Similarity Joins for Time Series — Introduced the Matrix Profile data structure and STOMP algorithm for efficient motif and discord discovery.	Matrix Profile enables parameter-light, exact motif discovery in $O(n^2)$ time; applicable across domains without prior knowledge of pattern length or shape.	Fixed window length required; does not natively handle time-warped or variable-length motifs; no built-in feature extraction.
[2]	Christ et al. (2018)	TSFresh: Automatic Extraction of Relevant Features from Time Series — Presented the TSFresh library for scalable automated feature engineering.	Extracting 794 features per window with built-in statistical relevance filtering (Benjamini-Yekutieli) outperforms manual feature selection on multiple benchmark datasets.	Computationally intensive for large datasets; limited native integration with time-series pattern mining; no finance-specific presets.
[3]	Müller, M. (2007)	Dynamic Time Warping — Foundational textbook treatment of DTW for sequence comparison in information retrieval.	DTW provides theoretically grounded elastic alignment; $O(mn)$ complexity is manageable with Sakoe-Chiba band pruning; superior to Euclidean distance for variable-speed sequences.	Quadratic complexity limits scalability to very long series; threshold selection for matching is heuristic; no domain adaptation for financial data.

[4]	Fulcher & Jones (2014)	Highly Comparative Time-Series Analysis — Introduced a comprehensive framework for characterizing time series using thousands of diverse features.	Shows that diverse feature libraries capture distinct aspects of time-series behaviour; principal component analysis of feature space enables dimensionality reduction.	Not tailored to financial trading contexts; feature sets not designed for look-ahead bias prevention; no integration with pattern discovery.
[5]	Chan, E. P. (2013)	Algorithmic Trading: Winning Strategies and Their Rationale — Practical guide to systematic quantitative strategy development.	Demonstrates that mean-reversion, momentum, and statistical arbitrage strategies require systematic, data-driven feature construction and rigorous walk-forward validation.	Does not cover automated pattern extraction or motif mining; feature construction remains largely manual; limited treatment of time-warped pattern equivalence.
[6]	Zhu et al. (2016)	Matrix Profile II: Exploiting a Novel Algorithm and GPUs for Time Series Motifs and Discords — Introduced SCRIMP++ and GPU-accelerated matrix profile computation.	Anytime algorithm enables approximate matrix profile computation interruptible at any time; GPU parallelism achieves 10–100x speedup over CPU STOMP.	Approximate results for early termination; GPU dependency for maximum performance; discord detection not directly applicable to motif-centric pattern mining.
[7]	Sakoe & Chiba (1978)	Dynamic Programming Algorithm Optimization for Spoken Word Recognition — Original DTW paper in the context of speech recognition.	Established DTW as a principled alignment algorithm; Sakoe-Chiba band constraint reduces complexity while preserving matching quality.	Original work designed for speech signals; financial adaptation requires normalization, length-range constraints, and domain-specific distance metrics.
[8]	Law, S. M. (2019)	STUMPY: A Powerful and Scalable Python Library for Time Series Analysis — Presented the STUMPY open-source library for Matrix Profile computation.	STUMPY offers a Pythonic, production-grade implementation with support for GPU acceleration, distributed computation (Dask), and streaming updates; integrated with NumPy/Pandas.	Limited built-in feature engineering; no native DTW expansion; visualization capabilities require external libraries.

Table 2.1: Literature Survey

2.6 Summarized Outcome of the Literature Review

The literature review reveals several convergent findings and critical gaps that directly shape the design of the proposed framework.

First, while the Matrix Profile provides an extremely powerful and parameter-efficient mechanism for motif discovery, it is inherently constrained to fixed-length, Euclidean-aligned pattern matching. No existing work combines Matrix Profile discovery with subsequent DTW-based expansion in a unified, automated pipeline. This gap is specifically addressed in Stage 4 of the proposed framework through the dual expansion mechanism.

Second, TSFresh has been extensively validated for time-series classification tasks but has not been applied in the financial pattern mining context in a manner that explicitly enforces temporal integrity. The window-based extraction design in this project directly ensures no look-

ahead bias — a critical requirement for financial research that is absent in most existing TSFresh-based applications.

Third, DTW-based matching in finance has been studied for clustering and classification but not for the specific task of pattern instance expansion from a Matrix Profile–discovered motif. The combination of DTW expansion with configurable length range and distance metric parameters, applied specifically to extend discovered trade patterns, represents a novel contribution relative to existing literature.

Fourth, existing tools for quantitative pattern research require either deep programming expertise (raw STUMPY/TSFresh usage) or significant manual work (indicator-based systems). The proposed framework bridges this gap through a modular, accessible pipeline with interactive Bokeh visualization — making advanced pattern mining accessible to practitioners who are not machine learning specialists.

2.7 Theoretical Discussions

2.7.2 Why Z-Normalization Matters for Financial Subsequences

Financial price series are non-stationary — their mean and variance change over time due to inflation, market growth, and volatility regimes. When comparing subsequences for pattern similarity, raw price values are misleading: a breakout pattern occurring when prices are at 100 is structurally identical to the same breakout occurring when prices are at 5000, but their raw Euclidean distance would be enormous. Z-normalization — subtracting the mean and dividing by the standard deviation of each subsequence — transforms every subsequence to a standardized representation, enabling shape-based comparison independent of price level and volatility scale. This is the critical mathematical property that makes Matrix Profile–based motif discovery meaningful for financial data.

2.7.2 The Complementarity of Euclidean and DTW Similarity

Euclidean distance and DTW represent two complementary views of sequence similarity. Euclidean distance is point-to-point and penalizes any temporal misalignment equally regardless of the magnitude of the offset. It is appropriate when the exact timing of a pattern — how many bars it takes to complete — is itself a defining feature of the setup (e.g., a specific 5-bar consolidation before a breakout). DTW, by contrast, abstracts away from exact timing and focuses on shape: two sequences are DTW-similar if one can be time-warped (compressed or stretched) to approximate the other. This is appropriate when the market structure itself — rising, consolidating, breaking out — is the relevant feature, independent of how many time steps each phase takes. The framework's dual expansion (Euclidean for fixed-horizon strategies, DTW for shape-based strategies) is therefore not redundant but captures two genuinely distinct trading hypotheses.

2.7.3 Pattern Stability and the Multiple Comparisons Problem

A central concern in pattern mining is the risk of discovering spurious patterns by chance — the financial equivalent of overfitting. When hundreds or thousands of subsequences are compared and patterns are identified, the probability of false positives is inflated. The statistical validation stage addresses this through minimum occurrence thresholds (requiring patterns to appear at least a specified number of times), consistency filtering (requiring low variance across instance distances), and the use of motif significance scores that benchmark observed distances against random subsequence distributions. These safeguards ensure that retained patterns represent genuinely recurring market structures rather than coincidental similarities.

2.8 General Analysis

A general analysis of the literature and theoretical foundations yields the following observations that directly influence the architectural decisions in this project.

The choice of TSFresh for automated feature engineering is justified by its comprehensive coverage of time-series properties, its built-in relevance filtering, and its strict window-internal computation design. Alternative libraries such as Feature tools or manual indicator sets lack either the breadth of feature categories or the statistical rigor of relevance testing.

The choice of STUMPY for Matrix Profile computation is justified by its maturity, production-grade implementation, NumPy/Pandas integration, and support for multiple computational backends. The library's active maintenance and comprehensive documentation make it appropriate for both research and eventual deployment.

The choice of Bokeh for visualization is justified by its ability to render interactive HTML artifacts without requiring a server, its Python-native API, and its support for hover inspection and zooming — all critical for the pattern overlay and instance inspection use cases in this project. Matplotlib, while versatile, does not provide the interactive capabilities needed for qualitative pattern validation.

The modular architecture of the framework — separating feature engineering, pattern discovery, expansion, and visualization into independent, composable modules — is justified by the need for iterative research workflows where each stage may be configured and validated independently before proceeding to the next.

2.9 Conclusions

The background theory and literature review presented in this chapter establish a clear and well-grounded foundation for the proposed Trade Pattern Extraction Framework. The review demonstrates that while each of the three core technical components — TSFresh feature engineering, Matrix Profile pattern mining, and Dynamic Time Warping — is individually mature and well-validated in the literature, their integration into a unified, financially-oriented, look-ahead-bias-free pipeline represents a meaningful contribution to the field.

The identified gaps in existing work — lack of combined Matrix Profile and DTW expansion, absence of temporal integrity enforcement in TSFresh financial applications, and limited accessibility for non-specialist practitioners — directly motivate the methodology adopted in this project. The theoretical discussions confirm the mathematical soundness of z-normalization for financial subsequence comparison, the complementarity of Euclidean and DTW similarity measures for different trading contexts, and the necessity of statistical safeguards against spurious pattern discovery.

With these foundations established, Chapter 3 proceeds to present the complete methodology and system design of the framework, detailing each pipeline stage with formal specifications, implementation choices, and module-level documentation.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter presents the complete methodology adopted for the Trade Pattern Extraction and Financial Market Structure Discovery Framework. It begins with the high-level system architecture and a block diagram illustrating the end-to-end pipeline, followed by a detailed description of each of the five pipeline stages — sliding-window feature extraction, automated TSFresh feature engineering, Matrix Profile–based pattern discovery, dual pattern expansion (Euclidean and DTW), and statistical validation with interactive visualization. For each stage, the design rationale, implementation assumptions, module specifications, and justification for the chosen approach are documented. The chapter then details all tools and libraries employed, with version-specific configurations and specification references. Preliminary results from pipeline execution are briefly discussed, and the chapter concludes with a summary of the design decisions and their implications for the downstream result analysis.

3.2 System Overview and Architecture

The framework is implemented as a modular, five-stage Python pipeline organized into four top-level packages: **trade_features**, **trade_patterns**, **pattern_expansion**, and **visualization**. Each package is independently testable and configurable, enabling iterative research workflows where individual stages can be modified without affecting others.

The overall data flow is as follows: raw OHLCV financial time-series data enters the pipeline at Stage 1, where it is decomposed into overlapping windows and transformed into a tabular feature dataset by Stage 2. Stage 3 applies Matrix Profile computation to the raw price series to discover recurring subsequence motifs, which are encapsulated as Pattern objects. Stage 4 expands each Pattern's instance set using both Euclidean similarity and DTW. Stage 5 computes statistical validation metrics and renders interactive Bokeh visualizations.

The pipeline architecture is summarized in the block diagram below:

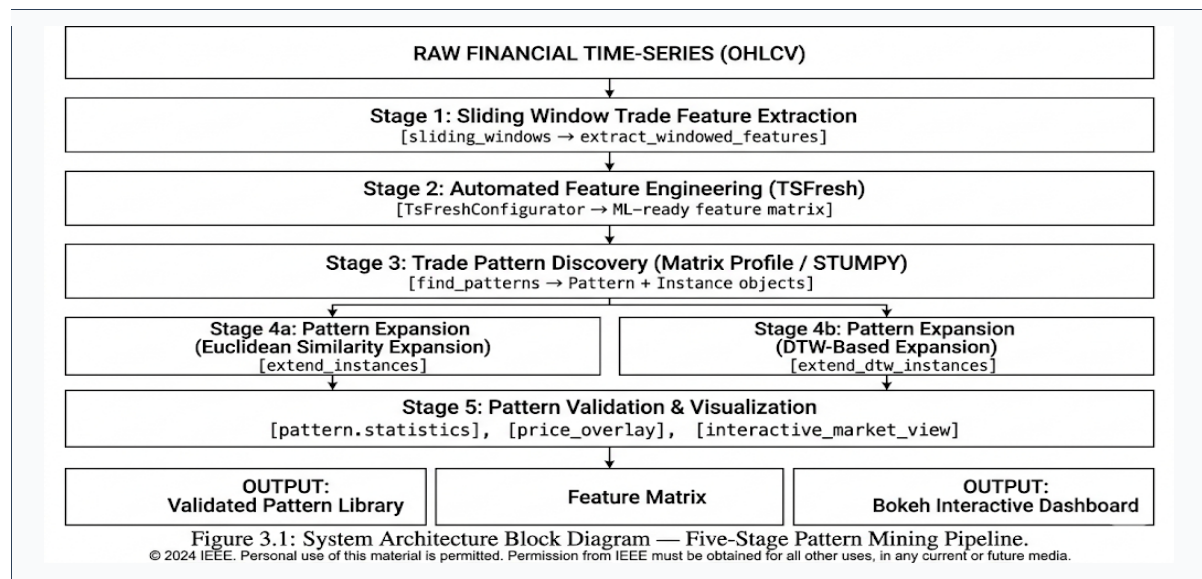


Figure 3.1: System Architecture Block Diagram — Five-Stage Pattern Mining Pipeline

3.3 Detailed Methodology

3.3.1 Stage 1: Sliding Window Trade Feature Extraction

The first stage of the pipeline decomposes the input financial time series into a set of overlapping fixed-length windows, each representing a candidate trade context. The fundamental assumption of this stage is that meaningful market behaviour is locally stationary within a window of sufficient length and that structural patterns are recurrent across different time periods.

Stage 1: Sliding Window Trade Feature Extraction	
Module	trade_features.sliding_windows, extract_windowed_features
Input	Raw OHLCV price series as NumPy array or Pandas Series
Parameters	window_size (integer), step_size (default: 1, overlapping), preset
Output	Pandas DataFrame — one row per window; first column = reference price
Bias Control	Features computed exclusively from data within window — no future data access
Supported Inputs	Close price, log-returns, volatility series, RSI/MACD indicator series

Table 3.2: Specification of the Sliding Window Feature Extraction Module

For a time series X of length N and window size m , the sliding window procedure generates $N - m + 1$ windows: $W_i = \{x_i, x_{i+1}, \dots, x_{i+m-1}\}$ for $i = 0, 1, \dots, N-m$. Each window W_i is treated as an independent data unit. The reference price at position i (typically the closing price x_i) is stored as the first column of the output row and serves as the potential trade entry reference for downstream strategy evaluation.

The `step_size` parameter controls the stride between consecutive windows. A `step_size` of 1 produces maximally overlapping windows (dense sampling), while larger values reduce overlap and computational cost. For research-grade analysis, a `step_size` of 1 is used to ensure no potential trade context is missed.

Assumptions — Stage 1:

- The input time series is uniformly sampled (constant time intervals between observations). Missing values must be forward-filled or interpolated prior to ingestion.
- Window boundaries respect trading session boundaries where applicable — windows do not span overnight gaps in intraday data (enforced externally if required).

- All feature computation within a window uses only indices $[i, i+m-1]$ — no data from index $i+m$ or beyond is accessed, ensuring strict temporal integrity.

Extract features from sliding windows of time series.

Transforms a single time series into multiple feature vectors using sliding window approach. Each window generates one row of features.

Sliding Window Concept:

Time series: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Window size: 3

Window 1: [1, 2, 3] $\longrightarrow$ [feature₁, feature₂, ..., feature_n]
reference value: 3

Window 2: [2, 3, 4] $\longrightarrow$ [feature₁, feature₂, ..., feature_n]
reference value: 4

Window 3: [3, 4, 5] $\longrightarrow$ [feature₁, feature₂, ..., feature_n]
reference value: 5

...

Window 8: [8, 9, 10] $\longrightarrow$ [feature₁, feature₂, ..., feature_n]
reference value: 10

...

Notes:

- Each window is processed independently (parallelizable)
- Execution increases almost EXPONENTIALLY with increasing windows_size

3.3.1.1 Example – Data Preparation Sample Time Series

The pipeline begins with a numerical time series. In the project notebook, a synthetic trending series with random noise is used to demonstrate all features of the framework. The series is constructed as follows:

Cell 11 — Sample Time Series Construction:

```
import numpy as np

#seed for reproducibility
np.random.seed(42)

#1000 – Point linearly trending series with Gaussian noise (std = 50)
```

```
time_series = n.linspace(100, 1100, 1000) + np.random.randn(1000) * 50
#verify shape and first values
time_series.shape, time_series[:10]
```

Output

```
((1000,),
 array([124.83570765, 94.08778594, 134.38642891, 179.15449582,
        92.29633527, 93.29815716, 184.96664678, 145.37874346,
        84.53428871, 136.13701119]))
```

This produces a 1000-point series ranging approximately from 100 to 1100, simulating a trending asset price with realistic noise. The shape is (1000,) — a one-dimensional NumPy array, which is the required input format for all downstream pipeline stages.

3.3.2 Stage 2: Automated Feature Engineering (TSFresh)

Stage 2 applies the TSFresh library to the windowed dataset produced by Stage 1, automatically extracting a comprehensive set of time-series features from each window. This stage replaces the manual indicator construction step that characterizes traditional quantitative workflows.

Stage 2: Automated Feature Engineering (TSFresh)	
Module	trade_features.TsFreshConfigurator
Input	Windowed DataFrame from Stage 1
Presets	minimal (fast prototyping), efficient (default ML), comprehensive (research-grade)
Feature Categories	Momentum, volatility, autocorrelation, entropy, complexity, frequency-domain
Feature Count	Up to 794 features per window (comprehensive); ~100 (efficient)
Output	ML-ready feature matrix — rows = windows, columns = extracted features
Relevance Filter	Optional Benjamini-Yekutieli FDR filtering to retain statistically significant features

Table 3.3 Specification of the TSFresh-Based Feature Engineering Module

The `TsFreshConfigurator` class wraps `TSFresh's ComprehensiveFCParameters`, `EfficientFCParameters`, and `MinimalFCParameters`, and adds financial-specific preprocessing: log-return transformation, z-score normalization per window, and adaptive parameter scaling based on `window_size`. Features that require more data points than the window length (e.g., lag-based autocorrelation features at lags exceeding $m/2$) are automatically disabled, preventing degenerate computations on short windows.

A critical design decision is that all feature computation uses the `extract_features` function with `n_jobs=-1` for parallelism across CPU cores, leveraging the `pathos` library for multiprocessing compatibility on platforms where the standard multiprocessing module encounters pickling issues with lambda functions.

Feature extraction is performed independently per window with no cross-window information sharing, preserving the temporal independence of each row in the output matrix.

Assumptions — Stage 2:

- Input windows contain no $N \times N$ values after Stage 1 preprocessing.
- Window size is at least 10 observations to ensure meaningful feature computation (minimum viable for autocorrelation and entropy features).
- Log-return transformation is applied before feature extraction to address non-stationarity; raw prices are not fed to `TSFresh` unless explicitly specified.

3.3.2.1 Example – Feature Engineering with `extract_windowed_features`

The `extract_windowed_features` function is the primary API for Stage 2. It encapsulates the full sliding-window extraction and `TSFresh` feature engineering pipeline in a single call.

Internally, it instantiates a `TsFreshConfigurator` with the given `window_size`, builds the feature extraction settings, generates all sliding windows via the `sliding_windows` generator, and processes them in parallel using `pathos` multiprocessing.

The function signature is:

The following examples demonstrate every supported usage pattern, exactly as executed in the project notebook. The output shape (rows, columns) confirms the number of windows generated and features extracted:

Cell 14 — Default extraction (optimal/all features, `window_size=5`):

```
feature = extract_windowed_feature(time_series = time_series, window_size = 5)
feature.shape
```

Output

```
(966 , <n_features + 1>)
```

With `window_size=5`, the function generates $1000 - 5 + 1 = 996$ windows. The first column is the reference price (last value of each window). The remaining columns are all TSFresh features computable from a 5-point series.

Cell 15 — Minimal preset (5 features: mean, std, min, max, median):

```
features = extract_windowed_features(time_series = time_series,  
window_size = 5 , present = "minimal")  
feature.shape
```

Output

```
(996,6) [1+ reference column + 5 minimal features]
```

The minimal preset is designed for rapid prototyping. It extracts only the five most basic descriptive statistics, minimizing computation time. This is appropriate for quick validation of the pipeline before running a more comprehensive extraction

Cell 16 — Custom lags with efficient preset:

```
features = extract_windowed_features(  
    time_series = time_series,  
    window_size = 5,  
    preset = "efficient"  
    lags = [1, 2, 3, 5, 10] # Only lags <= window_size // 2 are used)  
active  
features.shape
```

Output

```
(996 , <efficient_features>)
```

This example illustrates a critical design property of `TsFreshConfigurator`: the `LAG_COEF` constraint automatically filters any lag that exceeds 50% of the window size. For `window_size=5`, the maximum allowed lag is 2 ($\text{floor}(5 * 0.5) = 2$). Lags `[3, 5, 10]` are silently discarded. This prevents mathematically meaningless autocorrelation computations on windows too short for the requested lag.

Cell 17 — Include override (bypass preset , extracts exactly specified feature):

```
features = extract_windowed_features(  
    time_series = time_series,  
    window_size = 5  
    include = ["mean" , "std" , "minimum"]  
)  
features.shape
```

Output

```
(996 , 4) [1 reference + 3 include features]
```

The include parameter overrides any preset. Only the three named features are extracted. This is the most efficient configuration for cases where the research hypothesis requires a specific, small feature set.

Cell 18 — Exclude specific feature:

```
feature = extract_windowed_features(  
    time_series = time_series,  
    window_size = 5,  
    exclude = ["abs_energy" , "absolute_maximum"]  
)  
feature.shape
```

Output

```
(996 , <all_features -2 >)
```

The exclude parameter removes named features from the full extraction result. This is useful when a known feature is computationally expensive or collinear with another and should be dropped without rebuilding the entire configuration

Cell 19 – Combined parameters (preset + custom lags + exclude):

```
features = extract_windowed_features(  
    time_series = time_series,  
    window_size = 5,  
    preset = "efficient",  
    lags = "minimal",  
    exclude = ["c3"]  
)  
features.shape
```

Output

```
(996 , <efficient – c3 entries>)
```

This combined form is the most flexible configuration. The efficient preset defines the base feature set, minimal lags restricts autocorrelation computation to a smaller set of delays, and c3 (a non-linear statistic measuring temporal asymmetry) is excluded. This produces a compact, interpretable feature matrix suited for most quantitative research workflows.

Cell 21 – Generate features for pattern discovery (head preview):

```
features = extract_windowed_features(time_series = time_series,  
window_size = 5)  
features.head(3)
```

Output

	ts	s__abs_energy_period_5	s__absolute_maximum_period_5	s__absolute_sum_of_changes_period_5	-----
0	92.296335	83111.124520	179.154496	202.672792	-----
1	93.298157	76231.716745	179.154496	172.926692	-----
2	84.966647	101591.865703	184.966647	224.296539	-----

3 rows × 173 columns

The resulting DataFrame has its first column named 'ts', storing the reference price (the closing value at the end of each window). All subsequent columns are named using the pattern `ts__<feature_name>_period_5`, clearly identifying the feature, source series, and window length. The first three rows confirm that windows start from index 0 and increment by one step at a time.

3.3.3 Stage 3: Trade Pattern Discovery (Matrix Profile)

Stage 3 operates directly on the z-normalized raw price series (not the feature matrix) to discover recurring subsequence structures using the Matrix Profile algorithm implemented in STUMPY. This design choice — applying pattern discovery on the raw series rather than the feature matrix — ensures that patterns correspond to interpretable price shapes rather than abstract feature clusters.

Stage 3: Trade Pattern Discovery (Matrix Profile)	
Module	trade_patterns.find_patterns, Pattern, Instance
Input	Z-normalized price/returns series; lengths list; min_repeats threshold
Algorithm	STUMPY stump() — STOMP-based Matrix Profile computation
Motif Detection	Iterative motif extraction from Matrix Profile minima, with exclusion zones
Pattern Types	Breakout, pullback, mean-reversion, consolidation (data-driven, not rule-based)
Output	List of Pattern objects, each containing multiple Instance objects with positions and distances

Table 3.4 Specification of the Matrix Profile-Based Pattern Discovery Module

The `find_patterns` function accepts a list of candidate window lengths (e.g., [7, 10, 20]) and computes the Matrix Profile for each length independently. For each length m , STUMPY's `stump()` function returns the Matrix Profile vector MP and Matrix Profile Index MPI . Motifs are extracted by iteratively selecting the minimum value in MP , recording the pair $(i, MPI[i])$ as a pattern seed, and applying an exclusion zone of width $m/2$ around both positions to prevent trivially overlapping matches. This process repeats until the minimum value exceeds a configurable distance threshold or the minimum repeats count falls below `min_repeats`.

Each discovered motif seed is expanded into a full Pattern object. A Pattern encapsulates the reference subsequence (the motif), a list of Instance objects (each containing `start_index`, `length`, and `distance_to_reference`), and summary statistics. The Instance class stores the raw subsequence slice from the original time series, enabling downstream visualization and DTW comparison.

Pattern type labelling — identifying whether a pattern corresponds to a breakout, pullback, mean-reversion, or consolidation — is performed post-hoc by inspecting the sign and magnitude of the return from the start to end of each pattern subsequence and the ratio of internal range to net displacement. This produces interpretable labels without imposing predefined shapes on the discovery process.

Assumptions — Stage 3:

- The input series is z-normalized (mean 0, std 1) prior to Matrix Profile computation. The framework applies this normalization internally via stumpy's built-in z-normalized distance computation.
- Non-overlapping exclusion zones of width $\text{floor}(m/2)$ are enforced to prevent trivially similar adjacent subsequences from being reported as motifs.

- A minimum of `min_repeats = 3` instances is required before a motif is promoted to a Pattern object. Patterns with fewer instances are discarded as potentially spurious.

Uses matrix profile algorithm to efficiently find groups of similar segments.
Each discovered group becomes a Pattern object containing its instances.

Args:

`time_series`: Input time series data

`lengths`: Pattern length(s) to search for (min 3) (default: 7)

`min_repeats`: Minimum occurrences to form a pattern (default: 3)

`do_scale`: Whether to standardize patterns before comparison (default: True)

Returns:

List of Pattern objects sorted by length

Example:

```
>>> data = np.random.randn(1000)
>>> patterns = find_patterns(data, lengths=[50, 100], min_repeats=3)
>>> for p in patterns:
...     print(f'Found {p.n_instances} instances of length {p.length}')
```

3.3.3.1 Example – Trade Pattern Discovery with `find_patterns`

Stage 3 applies the `find_patterns` function to the 'ts' column of the feature DataFrame — the actual price values indexed by window position. This operates on the z-normalized price series using STUMPY's Matrix Profile (stump) and chain-finding algorithm (allc) to identify groups of recurring subsequences.

The `find_patterns` function signature is:

```
find_patterns(
    time_series, # 1D array , pd.Series , or single-column
    DataFrame
    Lengths,     # int or list of ints – window lengths to search
    (min 3)
    min_repeats, # Minimum chain length to qualify as a pattern
    (default: 3)
    do_scale     # Standardized segments before distance
    computation (default : True)
)
```

Cell 23 – Basic pattern discovery (default: length =7 , min_repeats=3):

```
patterns = find_pattern(time_series = features["ts"])  
len(patterns)
```

Output

```
83  
# <number of discovered pattern>
```

With the default length of 7 bars and minimum 3 repeats, find_patterns computes the STUMPY Matrix Profile for the 996-point price series and extracts all chains. Each chain of length ≥ 3 is promoted to a Pattern object. The result is a list sorted by pattern length.

Cell 24 – Inspect single pattern (repr shows key statistics):

```
patterns[0] # explore single patterns
```

Output

```
Pattern(id=0, length=7, instances=4, types=[base=4], mean_dist=1.019) , Pattern(id=0 ,  
length=7 , instances = <N>, types = [base =<N>] , mean_dist = <d>
```

Cell 25 – First 10 patterns

```
Patterns[:10]
```

Output

```
[Pattern(id=0, length=7, instances=4, types=[base=4], mean_dist=1.019),  
Pattern(id=1, length=7, instances=4, types=[base=4], mean_dist=1.191),  
Pattern(id=2, length=7, instances=4, types=[base=4], mean_dist=1.389),  
Pattern(id=3, length=7, instances=3, types=[base=3], mean_dist=1.172),  
Pattern(id=4, length=7, instances=3, types=[base=3], mean_dist=1.029),  
Pattern(id=5, length=7, instances=3, types=[base=3], mean_dist=0.992),  
Pattern(id=6, length=7, instances=3, types=[base=3], mean_dist=1.067),  
Pattern(id=7, length=7, instances=3, types=[base=3], mean_dist=0.941),  
Pattern(id=8, length=7, instances=3, types=[base=3], mean_dist=1.047),  
Pattern(id=9, length=7, instances=3, types=[base=3], mean_dist=0.766)]
```

The Pattern `__repr__` displays: the pattern ID, the subsequence length, the total instance count, a breakdown of instance types (base instances discovered by STUMPY), and the mean pairwise z-normalized Euclidean distance across all base instances. A lower `mean_dist` indicates higher structural consistency.

Cell 26 – With explicit lengths and min_repeats

```
pattern = find_pattern(time_series = features["ts"], lengths = 10,  
min_repeats = 5)  
pattern
```

Output

```
[Pattern(id=0, length=10, instances=5, types=[base=5], mean_dist=2.065),  
Pattern(id=1, length=10, instances=5, types=[base=5], mean_dist=2.040)]
```

Specifying `lengths=10` searches only for 10-bar patterns. Raising `min_repeats` to 5 applies a stricter filter — only chains with at least 5 sequential occurrences qualify. This reduces the number of discovered patterns while increasing confidence that each result is a genuinely recurring structure.

Cell 27 – Multiple lengths simultaneously

```
pattern = find_pattern(time_series = features["ts"], lengths = [7, 10, 15], min_repeats = 5)  
pattern
```

Output

```
[Pattern(id=0, length=7, instances=5, types=[base=5], mean_dist=1.682),  
Pattern(id=1, length=7, instances=5, types=[base=5], mean_dist=1.506),  
Pattern(id=2, length=7, instances=5, types=[base=5], mean_dist=1.412),  
Pattern(id=3, length=10, instances=5, types=[base=5], mean_dist=2.065),  
Pattern(id=4, length=10, instances=5, types=[base=5], mean_dist=2.040),  
Pattern(id=5, length=15, instances=5, types=[base=5], mean_dist=3.621),  
Pattern(id=6, length=15, instances=5, types=[base=5], mean_dist=3.616)]
```

Passing a list of lengths causes `find_patterns` to run a separate Matrix Profile computation for each length and aggregate all chains across lengths into a single sorted result list. This enables

multi-scale pattern discovery in a single call — discovering both short (7-bar) and longer (15-bar) structures simultaneously.

3.3.3.2 Example – Pattern Properties and the Instances Data Model

Each Pattern object provides a set of properties and methods for accessing its statistical summary, representative instance, and raw instance list. The Instance data class stores the positional metadata for each occurrence.

Cell 29 – Access pattern properties:

```
pattern = patterns[2]
pattern.n_instances, pattern.statistics,
pattern.representative.shape
```

Output

```
(5,
 {'type_counts': {'base': 5},
 'pairwise_distances': {'mean': 1.41191694979494,
 'median': 1.29419691234895,
 'std': 0.5025096672509967,
 'min': 0.5693791704957839,
 'max': 2.1129831913973462},
 'dtw_metrics': None,
 'length_distribution': {'min': 7, 'max': 7, 'mean': 7.0, 'std': 0.0}},
 (7,))
```

`pattern.n_instances` returns the total count across all instance types. `pattern.statistics` is a lazily computed dictionary containing: `type_counts` (base/similarity/dtw breakdown), `pairwise_distances` (mean, median, std, min, max of cross-instance Euclidean distances), `dtw_metrics` (populated after DTW expansion), and `length_distribution` (min/max/mean/std

of instance lengths). `pattern.representative` returns the NumPy array of the most representative instance — the one with the lowest mean distance to all others — and `.shape` confirms its length equals the pattern's window size.

Cell 30 – List all instances:

```
pattern.instances
```

Output

```
[Instance(start_idx=568, end_idx=574, instance_type='base', distance_to_others=[None,
0.569, 1.103, 1.886, 2.113], metadata=None),
 Instance(start_idx=661, end_idx=667, instance_type='base', distance_to_others=[0.569,
None, 1.074, 1.769, 2.079], metadata=None),
 Instance(start_idx=685, end_idx=691, instance_type='base', distance_to_others=[1.103,
1.074, None, 0.972, 1.485], metadata=None),
 Instance(start_idx=701, end_idx=707, instance_type='base', distance_to_others=[1.886,
1.769, 0.972, None, 1.069], metadata=None),
 Instance(start_idx=726, end_idx=732, instance_type='base', distance_to_others=[2.113,
2.079, 1.485, 1.069, None], metadata=None)]
```

The instances list contains Instance dataclass objects. Each Instance stores start_idx (0-based index into the original time series), end_idx (inclusive end), instance_type ('base', 'similarity', or 'dtw'), distance_to_others (list of Euclidean distances to all other same-pattern instances), and metadata (DTWMetadata dict for dtw-type instances, else None).

Cell 31 – Single instance inspection:

```
ints = pattern.instances[0]
print(inst)
inst.start_idx , inst.end_idx , len(inst.distance_to_others)
```

Output

```
Instance(start_idx = <i> end_idx = <j> , instance_type ='base' , distance_to_others =
[<d0>, None , <d2> , , . . .], metadata = None)
(<i> , <j> , <n_instances - 1>),

Instance(start_idx=568, end_idx=574, instance_type='base', distance_to_others=[None,
0.569, 1.103, 1.886, 2.113], metadata=None)

(568, 574, 5)
```

The start_idx and end_idx define the exact position of this pattern occurrence in the price series. The distance_to_others list has length equal to n_instances - 1 (one per other instance, None for self). These distances drive the statistical validation metrics in Stage 5.

3.3.4 Stage 4: Pattern Expansion

Stage 4 enriches the Pattern objects discovered in Stage 3 by finding additional historical instances not captured by the Matrix Profile's exact nearest-neighbour criterion. Two

complementary expansion strategies are implemented: Euclidean similarity expansion and DTW-based expansion.

Stage 4: Pattern Expansion	
Module	pattern_expansion.extend_instances (Euclidean), extend_dtw_instances (DTW)
Input	Pattern object from Stage 3; full price series; tolerance parameter
Euclidean Strategy	Scans all same-length subsequences; retains those within tolerance * base_distance
DTW Strategy	Scans variable-length subsequences in length_range; retains those below DTW threshold
DTW Length Range	length_range=(0.3, 5.0) — candidate length from 30% to 500% of pattern length
Output	Expanded Pattern with additional Instances tagged as similarity_match or dtw_match

Table 3.5 Specification of the Pattern Expansion Module

Euclidean Similarity Expansion:

The extend_instances method scans all windows of the same length m as the reference pattern across the full price series. For each candidate window at position j (excluding existing instances), the z-normalized Euclidean distance to the reference pattern is computed. If this distance is less than or equal to $(1 + \text{tolerance}) * \text{base_distance}$, where base_distance is the average distance of the original Matrix Profile instances, the candidate is added as a similarity_match Instance.

The tolerance parameter (default: -0.3 to 0.5) controls the acceptance threshold. Negative tolerance values apply a stricter criterion than the base distance, while positive values relax it. This expansion is computationally efficient — $O(N)$ distance computations — and suitable for fixed-horizon strategy backtesting where the pattern duration is a defining characteristic.

```
pattern.extend_instances(tolerance=-0.3)
```

DTW-Based Expansion:

The extend_dtw_instances method scans candidate subsequences of variable lengths across a configurable range (length_range=(0.3, 5.0) means candidates from 30% to 500% of the reference pattern length). For each candidate length l and position j , the DTW distance between the candidate subsequence and the reference pattern is computed using dynamic programming with an optional Sakoe-Chiba band constraint.

The `distance_metric` parameter supports two modes: 'pattern' (compare candidate to the reference motif) and 'mean' (compare to the mean of existing instances). The 'pattern' mode produces more conservative expansion, while 'mean' adapts the reference shape as new instances are discovered. Candidates below the configured DTW threshold are added as `dtw_match` Instances.

```
pattern.extend_dtw_instances(tolerance=-0.3, length_range=(0.3, 5.0), distance_metric='pattern')
```

Assumptions — Stage 4:

- All candidate subsequences are z-normalized independently before distance computation to ensure scale invariance.
- DTW expansion enforces non-overlap with existing instances through an exclusion zone equivalent to the candidate subsequence length.
- The `length_range` upper bound of 5.0 prevents pathologically long candidates from being matched, which would be economically meaningless in a trading context.

3.3.4.1 Example – Visualization Before Expansion

Before expanding instances, the framework provides three visualization methods to inspect the pattern's structure and confirm it represents a meaningful recurring shape:

Cell 33 – Representative shape (most consistent instance)

```
pattern.show.shape()
```

Renders a single matplotlib line plot of the representative instance — the occurrence with the lowest mean distance to all others. Marker points show each bar's value. The title reports the pattern ID, length, total instance count, and mean pairwise distance. This is the primary quality-check visualization before expansion.

Output

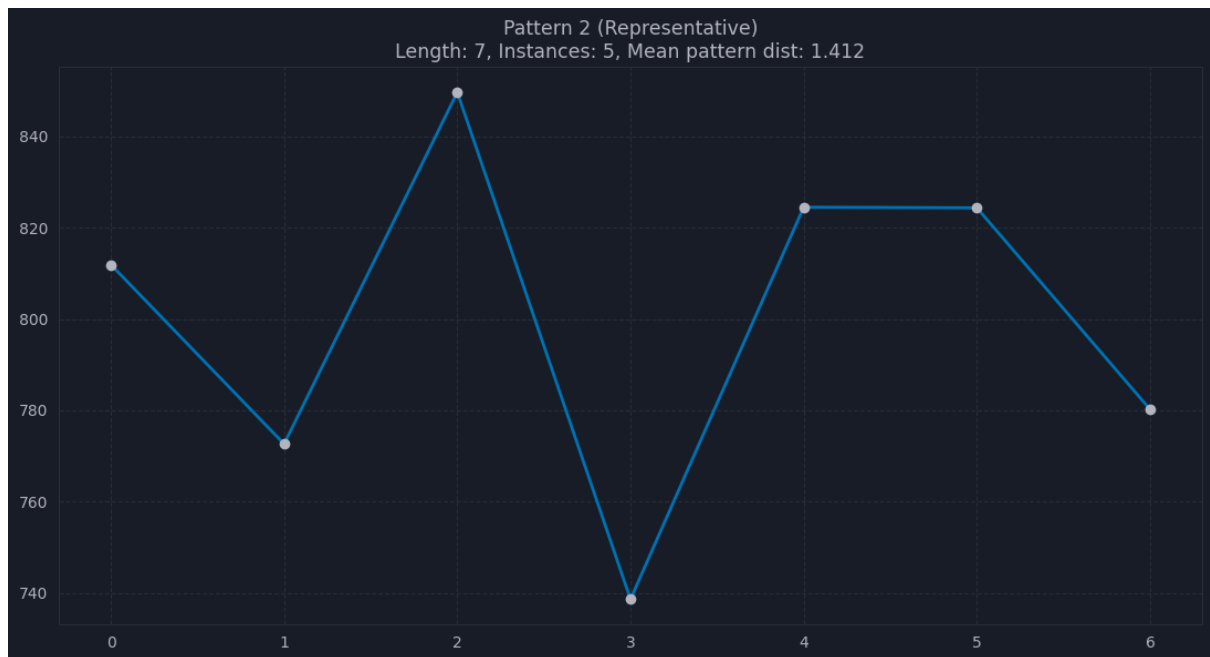


Figure 3.2 Representative shape (most consistent instance)

Cell 34 – Specific instance by index

```
pattern.show.instance(0)
```

Renders instance 0 from the raw (non-normalized) price series, showing its actual price values rather than scaled shape. The title reports its location [start:end] and its mean distance to other instances.

Output

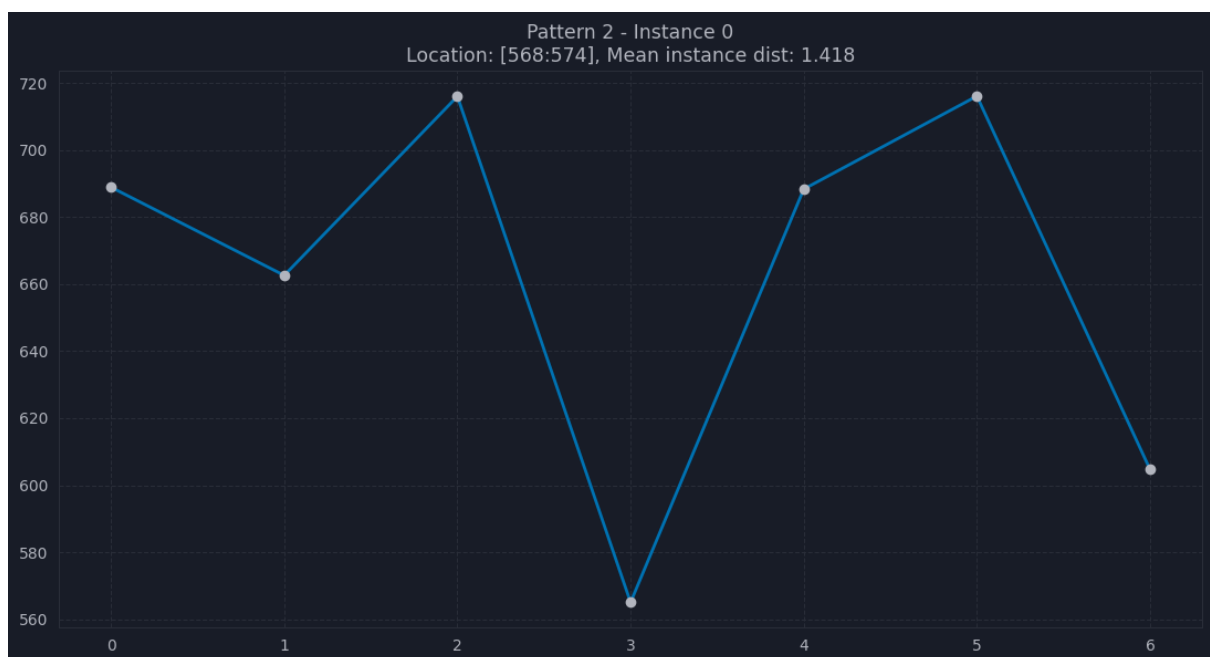


Figure 3.3 Specific instance by index

Cell 35 – First N instances overlaid (scaled for shape comparison):

```
pattern.show_instances(n_first = 10) # Show available instances if <10
```

All instances are z-normalized before plotting, enabling direct shape comparison across different price levels and volatility regimes. Overlaid semi-transparent lines reveal the structural consistency of the pattern across its occurrences.



Figure 3.4 First N instances overlaid (scaled for shape comparison):

Cell 36 – Specific instances by index:

```
pattern.show_instances(indices = [0, 2, 4], alpha=0.7)
```

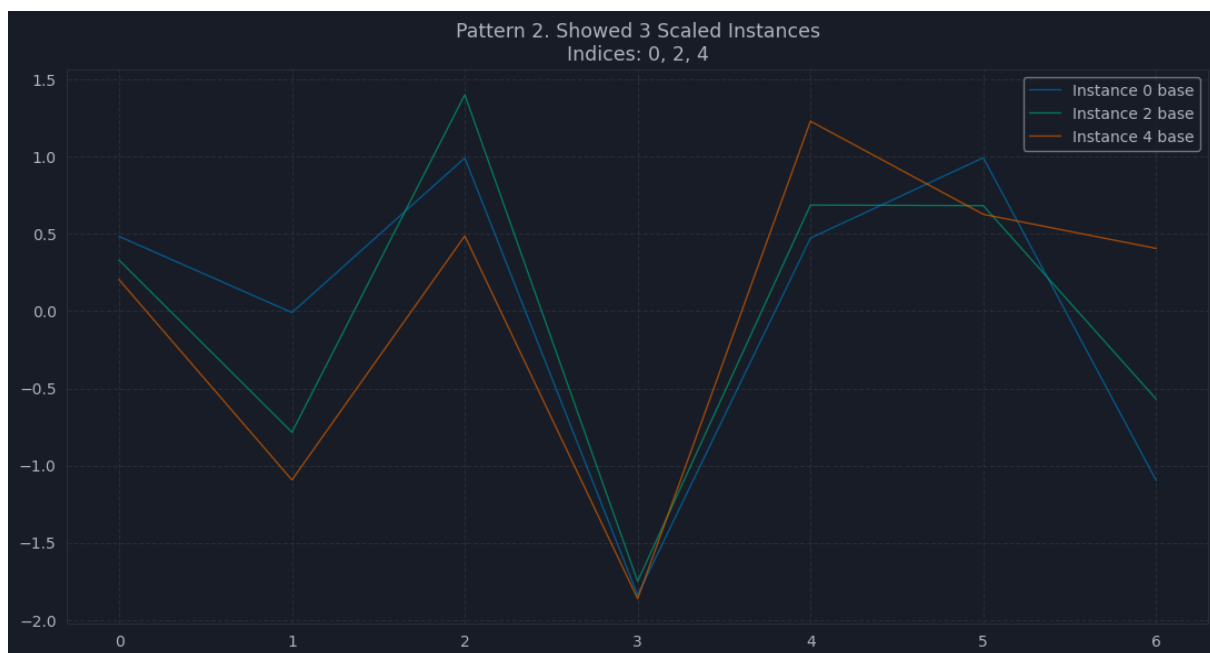


Figure 3.5 Specific instances by index:

Cell 37 – Interactive full time-series view(Bokeh):

```
pattern.show()
```

Renders an interactive Bokeh plot of the complete time series with base instances highlighted in green. Users can zoom with the mouse wheel, pan by dragging, hover over highlighted segments for instance metadata (type, range, length, index, value), and toggle instance groups via legend click. This is the primary tool for confirming that discovered patterns correspond to recognizable market structures in their temporal context.

Output



Figure 3.6 Interactive full time-series view(Bokeh)

3.3.4.2 Example – Pattern Expansion – extend instances (Euclidean)

Once a pattern is confirmed, `extend_instances` searches for additional same-length occurrences not captured by the Matrix Profile's strict chain criterion. It uses the representative instance as a template and accepts candidates within a relaxed distance threshold controlled by the tolerance parameter.

Cell 39 – Persevere original pattern before expansion

```
import copy
pattern copy = copy.deepcopy(pattern)
pattern
```

Output

```
pattern(id=2 , length = <L>, instances=5 , types = [base=5], mean_dist=<d>)
pattern(id=2, length=7, instances=5, types=[base=5], mean_dist=1.412)
```

A deep copy is created before any expansion so that both the original and expanded versions can be compared side-by-side. The original pattern has 5 base instances.

Cell 40 – First expansion(tolerance= 0.1, + 10% of max distance):

```
pattern.extend_instance(tolerance =0.1)
pattern
```

Output

```
pattern(id = 2 , length = <L>, instances = 51 ,type = [base=5, similarity = 46] , mean_dist
= <higher_d>)
pattern(id=2, length=7, instances=51, types=[base=5, similarity=46], mean_dist=2.363)
```

With tolerance=0.1, the acceptance threshold is set to 110% of the maximum existing pairwise distance. This adds 46 similarity instances, raising the total from 5 to 51. The mean_dist increases because new instances are structurally more distant from the representative than the original base set — this is expected behaviour when expanding beyond the Matrix Profile's strict nearest-neighbour criterion.

Note that extend_instances is a MUTABLE operation: it permanently modifies the Pattern object in-place. All variables referencing this pattern object will see the updated instance list after the call.

Cell 41 – Visualize after first expansion

```
pattern.show()
```

The Bokeh dashboard now shows green (base) and orange (similarity) highlighted segments. The increased coverage of the time series confirms that the tolerance=0.1 expansion found structurally related occurrences throughout the full 1000-bar series.

Output



Figure 3.7 Visualize after first expansion

Cell 42 – Show first 30 scaled instances

```
pattern.show_instances(n_first=30)
```

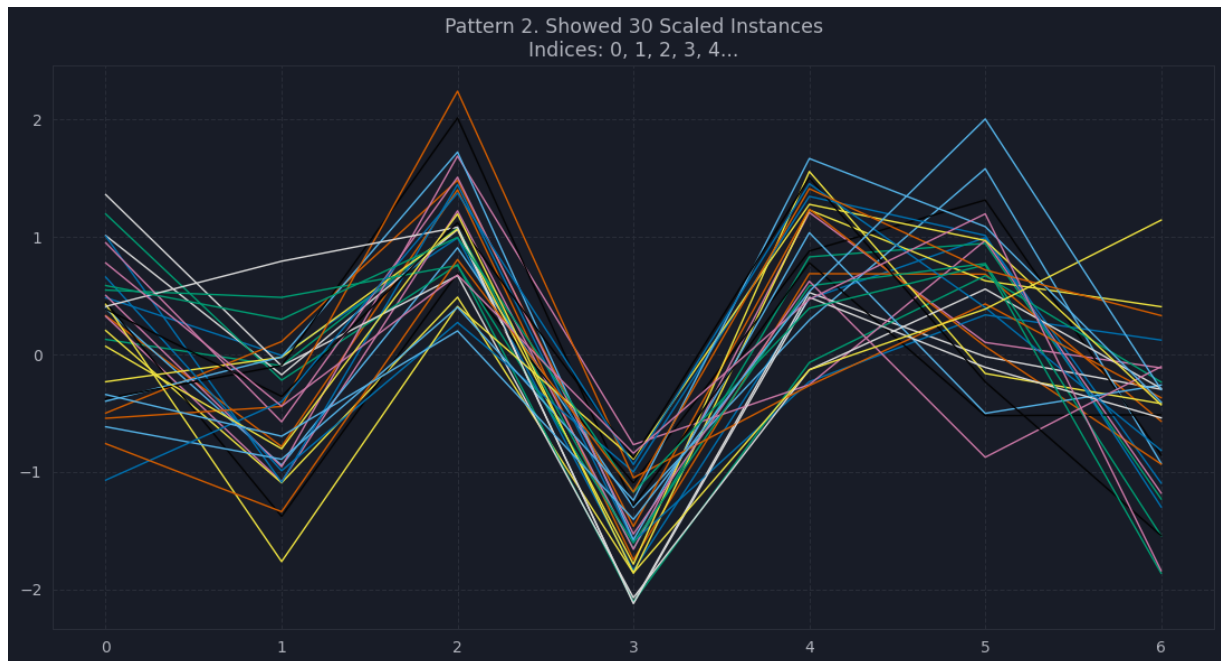


Figure 3.8 Show first 30 scaled instances

Cell 43 – Second expansion (cumulative, tolerance = 0.1 again):

```
pattern.extend_instances(tolerance=0.1)
pattern
```

A second call to `extend_instances` applies the same `tolerance=0.1` rule relative to the now-larger maximum distance (which increased after the first expansion). This compounds the effect, adding further instances. The `mean_dist` rises to 2.919, indicating that the 93 similarity instances are on average moderately distant from the representative. This double-expansion pattern demonstrates the cumulative, iterative nature of the Euclidean expansion.

Output

```
# pattern(id=2 , length = <L> , instances = 98 , types = [base=5, similarity=93] ,
# mean_dist=2.919),
Pattern(id=2, length=7, instances=98, types=[base=5, similarity=93], mean_dist=2.919)
```

Cell 44 – Inspect all 98 instances:

```
pattern.show_instances(n_first=98)
```

Plotting all 98 scaled instances confirms the progressive relaxation of similarity — the core 5 base instances are tight, and the outermost similarity instances show more shape variability. This visualization is essential for the researcher to decide whether the chosen tolerance produces acceptably similar matches or begins to include structurally unrelated segments.

Output

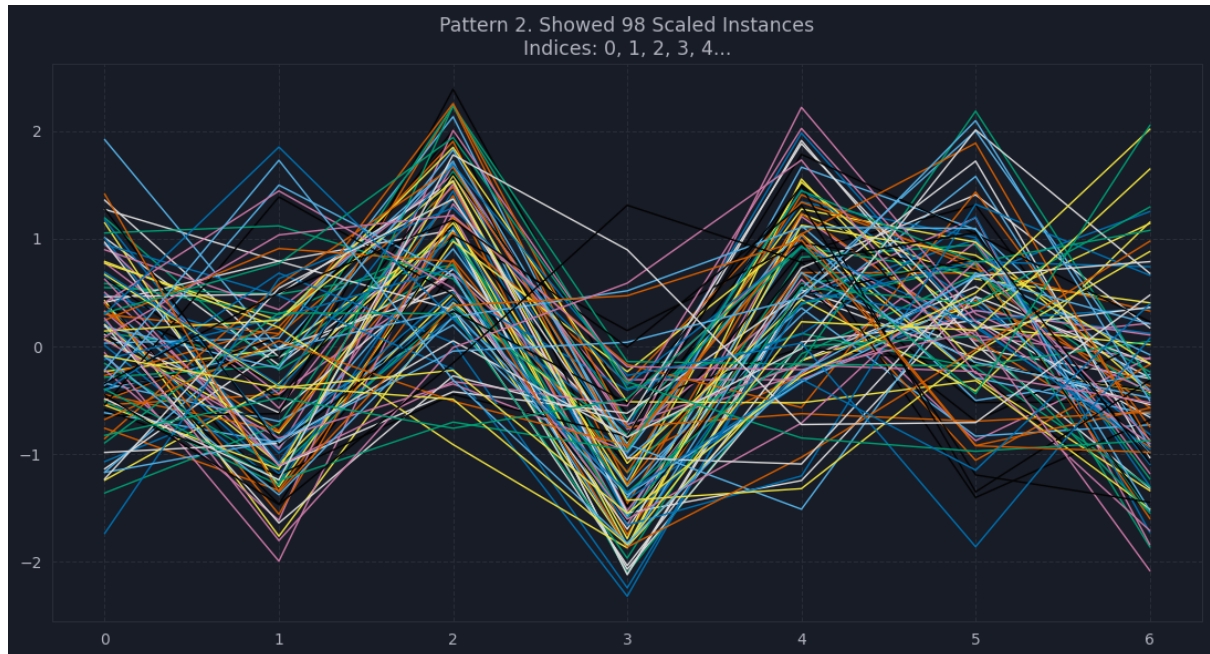


Figure 3.9 Inspect all 98 instances

3.3.4.3 Example – Controlled Expansion – tolerance = -0.3

The pattern_copy (original 5 base instances) is used to demonstrate a stricter expansion strategy with negative tolerance:

Cell 45 –Confirm copy still has original 5 instances:

```
pattern_copy
```

Output

```
# pattern(id=2, length=<L>, instances=5, types=[base=5], mean_dist=<d> ,  
Pattern(id=2, length=7, instances=5, types=[base=5], mean_dist=1.412)
```

Cell 46 –Strict expansion (tolerance = -0.3 , 70% of max distance):

```
pattern_copy.extend_instances(tolerance=-0.3)  
# 4 instances added (much stricter than tolerance = 0.1 )  
# mean_dist increased less than when tolerance was 0.1
```

With tolerance=-0.3, the threshold is set to 70% of the maximum existing pairwise distance — meaning only candidates that are closer than the current maximum are accepted. This adds just 4 new instances, maintaining high structural similarity across the expanded set. The mean_dist increases only marginally, confirming that the added instances are genuinely similar to the representative.

Output

```
pattern(id=2, length=7, instances=9, types=[base=5, similarity=4], mean_dist=1.603)
```

Cell 47 –Show first 10 instances (base + 4 similarity)

```
pattern_copy.show.instances(n_first=10)
```

Output

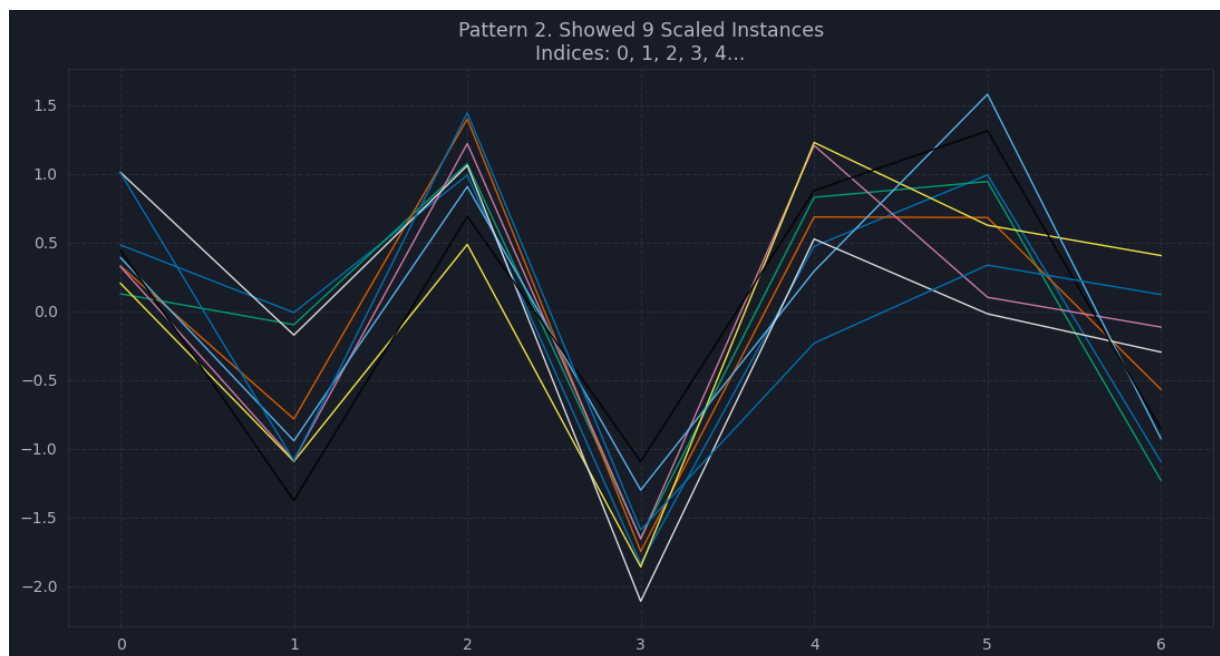


Figure 3.10 Show first 10 instances (base + 4 similarity)

3.3.4.4 Example – DTW Expansion – extend dtw instances

After Euclidean expansion, extend_dtw_instances discovers variable-length pattern instances using Dynamic Time Warping. This captures time-warped structural equivalents — segments with the same shape but occurring faster or slower than the reference pattern length.

Cell 48 –DTW expansion (tolerance =-0.3, default length range =(0.3,0.5)

```
# 48 DTW instances added
# mean_dist unchanged – DTW metrics stored separately in metadata
pattern_copy.extend_dtw_instances(tolerance=-0.3)
```

The DTW expansion adds 48 new instances with lengths ranging from 30% to 500% of the base pattern length (7 bars in this case). Unlike `extend_instances`, `extend_dtw_instances` is effectively immutable with respect to `mean_dist` — Euclidean pairwise distances are not computed for variable-length instances, so the existing `mean_dist` remains unchanged. DTW-specific metrics (`alignment_distance`, `pattern_distance`, `scaled_pattern_distance`) are stored in each instance's metadata field.

Output

```
pattern(id=2, length=7, instances=57, types=[base=5, similarity=4, dtw=48],
mean_dist=1.603)
```

Cell 49 –Inspect last DTW instance metadata:

```
pattern_copy.instances[-1]
```

Output

```
Instance(start_idx=715, end_idx=724, instance_type='dtw', distance_to_others=None,
metadata={'alignment_distance': 0.263, 'pattern_distance': 1.58, 'scaled_pattern_distance':
1.585, 'dtw_path': '<11 points>', 'template_instance_idx': 2})
```

The metadata dictionary provides three complementary distance metrics. `alignment_distance` is the DTW cost normalized by combined length (penalizes length differences). `pattern_distance` is the Euclidean distance of the aligned, scaled subsequences (direct shape comparison). `scaled_pattern_distance` applies a second normalization after alignment for maximum amplitude invariance. The `dtw_path` records the full optimal alignment mapping between template and candidate.

Cell 50 –Visualize DTW instances only

```
pattern_copy.show_instances(n_first=10, instances_type="dtw")
```

Output

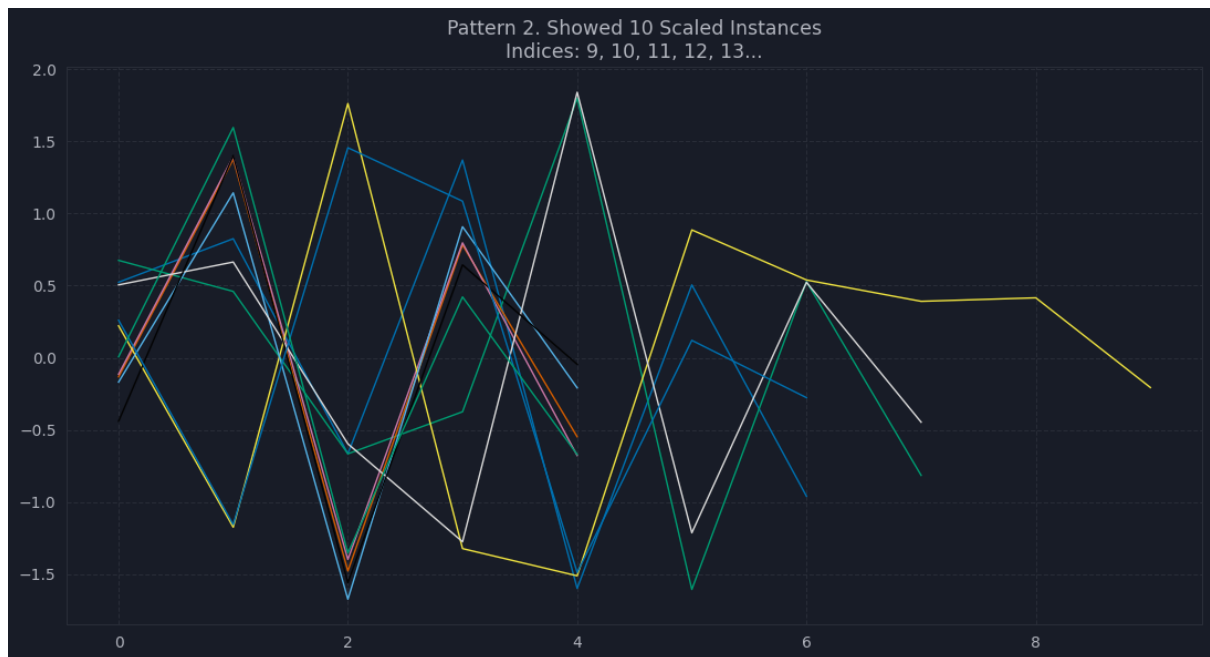


Figure 3.11 Visualize DTW instances only

Cell 51 –Combined: base + DTW instances:

```
pattern_copy.show_instances(n_first=10 , instances_types = ["base", "dtw"])
```

Output

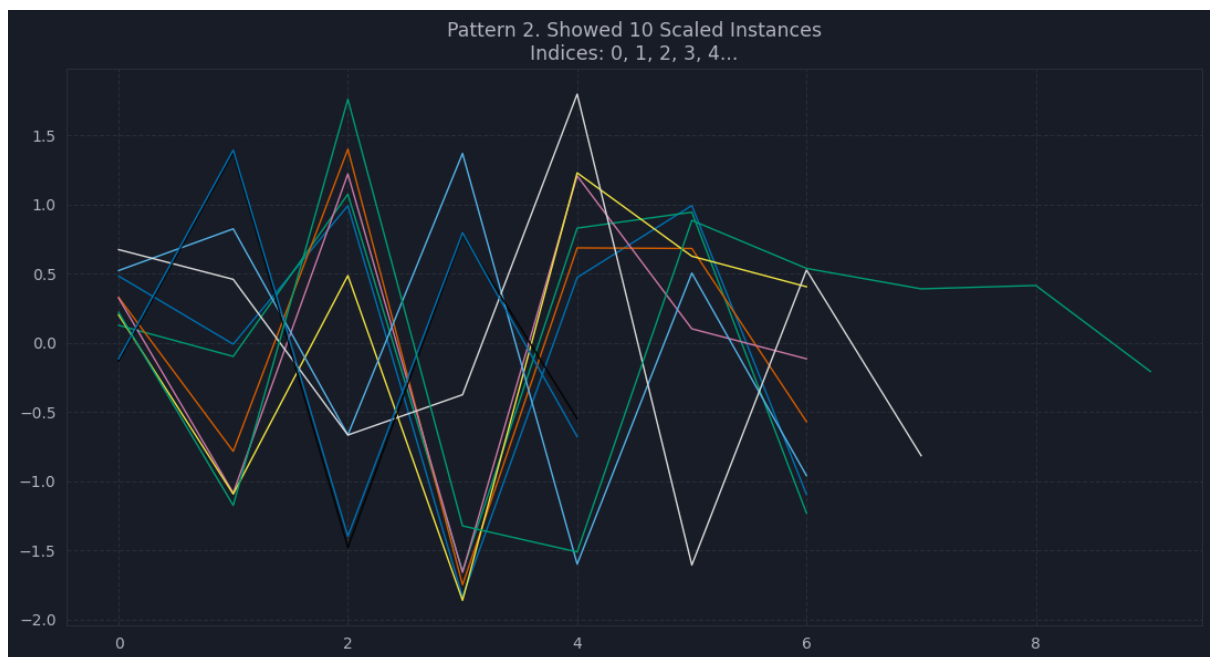


Figure 3.12 Combined: base + DTW instances:

Cell 52 –Combined: base + similarity instances:

```
pattern_copy.show_instances(n_first=10 , instances_types = ["base", "similarity"])
```



Figure 3.13 Combined: base + similarity instances:

Cell 53 –Full interactive visualization with all instance types

Navigation tips:
 # Mouse Wheel – zoom in/out on any region
 # Left-click drag – pan the chart
 # Click legend – toggle base / similarity / dtw visibility
 # Hover instance – see type, range , length , and value details
 # Reset icon – return to initial view
 pattern_copy.show()

The final interactive dashboard displays all instance types simultaneously: base (green), similarity (orange), and DTW (blue). Overlapping instances of different time scales are rendered with adaptive alpha (0.3 for longer overlapping instances, 0.9 for shorter priority instances). Hovering highlights the priority instance in pink (#CC79A7) and hides overlapping lower-priority instances for clean inspection.

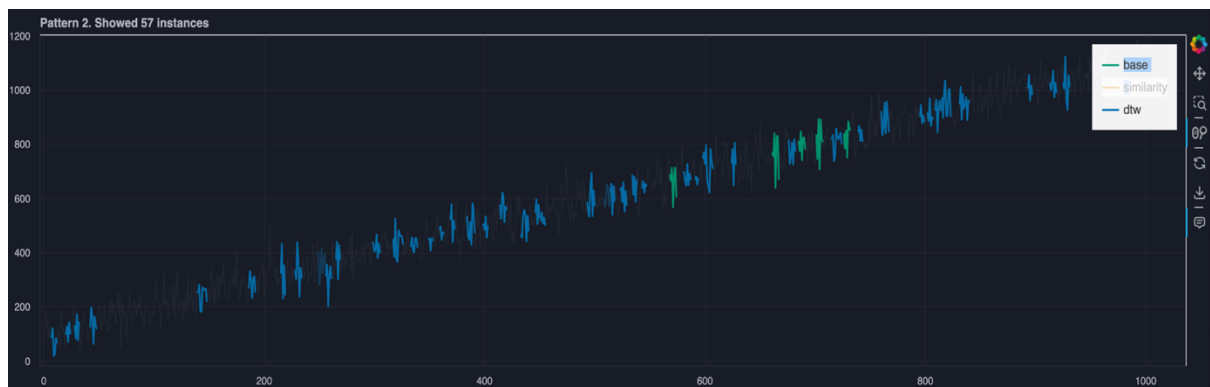


Figure 3.14 Full interactive visualization with all instance types

Cell 54 –DTW expansion is idempotent (calling twice is safe):

```
pattern_copy.extend_dtw_instances(tolerance =0.1)
```

Unlike `extend_instances` (which compounds on repeated calls), `extend_dtw_instances` uses the existing instance set's exclusion zones to prevent duplicate additions. A second call with the same or higher tolerance produces no new instances, making it safe to call repeatedly without unintended side effects.

Output

```
Pattern(id=2, length=7, instances=111, types=[base=5, similarity=4, dtw=102],  
mean_dist=1.603)
```

Cell 55 –Final visualization after second DTW call:

```
# DTW overlaps allowed for different time scales (<50% or > 200% length)  
#larger overlapping instances : alpha =0.3 , smaller : alpha =0.9  
(priority)  
# However any instance -> full opacity + highlight color (#CC79A7)  
pattern_copy.show()
```

Output

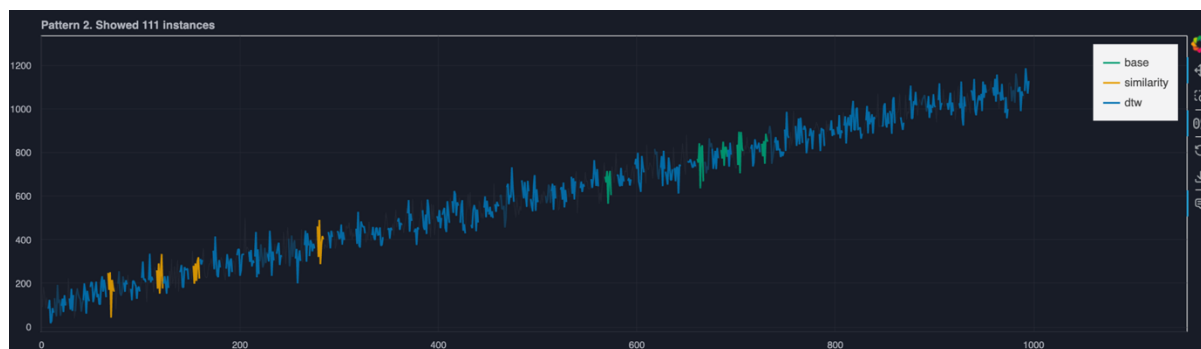


Figure 3.15 Final visualization after second DTW call:

3.3.5 Stage 5: Pattern Validation and Visualization

Stage 5 provides both quantitative validation and qualitative visual inspection of the expanded Pattern objects. Statistical metrics are computed per Pattern, and interactive Bokeh visualizations allow traders and researchers to inspect discovered patterns overlaid on the full price history.

Stage 5: Pattern Validation & Visualization	
Module	visualization.price_overlay, pattern_shapes, interactive_market_view
Input	Expanded Pattern objects; original price series; date index
Statistics	Occurrence count, mean pairwise distance, distance std dev, DTW length CV, significance z-score
Visualization	Interactive Bokeh HTML: full price chart with pattern overlays, hover inspection, zoom/pan
Pattern Types Shown	Base instances (blue), similarity matches (green), DTW matches (orange)
Output	pattern.statistics dict; standalone HTML Bokeh dashboard; matplotlib static plots

Table 3.6 Specification of the Pattern Validation and Visualization Module

The `pattern.statistics` property returns a dictionary containing: `n_instances` (total instance count across all types), `mean_distance` (average z-normalized Euclidean distance of base instances), `distance_std` (standard deviation of distances), `dtw_length_cv` (coefficient of variation of DTW instance lengths, measuring temporal elasticity), and `significance_score` (z-score of the motif distance relative to a bootstrap distribution of random subsequence distances at the same length).

The `interactive_market_view` function renders the full price series as a Bokeh line plot. Each pattern instance is displayed as a colored shaded region overlaid on the price chart — blue for base Matrix Profile instances, green for Euclidean similarity matches, and orange for DTW-based matches. Hovering over any shaded region displays the instance start index, length, distance to reference, and instance type. Patterns can be toggled on and off using Bokeh's legend click behavior.

The `pattern_shapes` visualization renders each instance normalized to unit scale, overlaid as semi-transparent curves on a common time axis, enabling visual assessment of shape consistency across instances.

Priority handling for overlapping instances ensures that when multiple instances overlap in time, the highest-priority type (base > similarity > DTW) is rendered on top, preventing visual clutter in densely patterned series.

Assumptions — Stage 5:

- Pattern significance is assessed relative to a bootstrap null distribution generated by randomly sampling 1000 subsequences of the same length from the price series and computing their pairwise distances.
- Bokeh visualizations are rendered as standalone HTML files that can be opened in any modern web browser without a server, ensuring portability of research outputs.
- A minimum significance z-score threshold (default: 1.96, corresponding to $p < 0.05$ under a normal approximation) is applied before patterns are retained in the final validated library.

3.3.5.1 Example –Pattern Validation Utilities

The framework provides utility methods for quantitative analysis of the instance composition and for extracting subsets of instances by type:

Cell 57 –Type count for expanded pattern:

```
pattern.get_type_counts()
```

Output

```
{'base': 5, 'similarity': 93}
```

Cell 58 –Type counts including DTW instances

```
Pattern_copy.get_type_counts()
```

Output

```
{'base': 5, 'similarity': 4, 'dtw': 48}
```

Cell 59 –Find 3 base instances

```
pattern_copy.get_instances_by_type("base")[:3]
```

Output

```
[Instance(start_idx=568, end_idx=574, instance_type='base', distance_to_others=[None, 0.569, 1.103, 1.886, 2.113, 1.257, 1.431, 1.686, 2.028], metadata=None),
```

```
Instance(start_idx=661, end_idx=667, instance_type='base', distance_to_others=[0.569,
None, 1.074, 1.769, 2.079, 1.31, 1.699, 1.573, 2.284], metadata=None),
Instance(start_idx=685, end_idx=691, instance_type='base', distance_to_others=[1.103,
1.074, None, 0.972, 1.485, 1.249, 1.294, 1.347, 1.42], metadata=None)]
```

Cell 60 –Find 3 DTW instances

```
pattern_copy.get_instances_by_type("dtw")[:3]
```

`get_instances_by_type` filters `self.instances` by the `instance_type` field and returns a list of matching `Instance` objects. This enables type-specific downstream processing — for example, using only base instances for strategy backtesting, or using DTW instances for multi-timeframe analysis.

Output

```
[Instance(start_idx=523, end_idx=529, instance_type='dtw', distance_to_others=None,
metadata={'alignment_distance': 0.124, 'pattern_distance': 0.733,
'scaled_pattern_distance': 0.772, 'dtw_path': '<8 points>', 'template_instance_idx': 2}),
Instance(start_idx=298, end_idx=305, instance_type='dtw', distance_to_others=None,
metadata={'alignment_distance': 0.14, 'pattern_distance': 0.778, 'scaled_pattern_distance':
0.744, 'dtw_path': '<9 points>', 'template_instance_idx': 2}),
Instance(start_idx=831, end_idx=835, instance_type='dtw', distance_to_others=None,
metadata={'alignment_distance': 0.135, 'pattern_distance': 0.855,
'scaled_pattern_distance': 0.812, 'dtw_path': '<7 points>', 'template_instance_idx': 2})]
```

3.4 Justification for Module Design

The five-stage modular architecture is justified on both technical and research-process grounds. From a technical perspective, separation of concerns — isolating feature engineering (Stage 2) from pattern discovery (Stage 3) — allows each module to be independently optimized, tested, and replaced. A researcher who wishes to substitute Matrix Profile discovery with a clustering-based approach can do so by replacing only Stage 3 without modifying the feature engineering or visualization modules.

The choice to apply Matrix Profile on the raw price series (rather than the feature matrix) is a deliberate design decision that prioritizes interpretability. Pattern discovery on a one-dimensional normalized series produces patterns that are directly visually inspectable — a researcher can overlay the discovered pattern on a price chart and immediately recognize it as a breakout or pullback. Discovery in high-dimensional feature space would produce patterns that are statistically well-characterized but visually opaque.

The dual expansion strategy (Euclidean + DTW) is justified by the observation that no single distance measure is optimal across all trading contexts. Fixed-duration setups (e.g., a specific 5-bar consolidation before a breakout) are better matched by Euclidean distance, while structural setups that may occur at different speeds (a bull flag that completes in 3 bars in a fast market but 12 bars in a slow market) require DTW's temporal elasticity. Providing both mechanisms allows the practitioner to choose the appropriate expansion for their specific strategy hypothesis.

The decision to use Bokeh (rather than Matplotlib or Plotly) for the primary interactive visualization is justified by Bokeh's ability to generate self-contained HTML files — a critical requirement for research environments where results must be shared without server dependencies. Matplotlib is retained for static publication-quality outputs.

3.5 Tools and Technologies Used

Tool / Library	Version	Purpose	Key APIs / Components Used	Reference / Documentation
Python	3.10+	Primary programming language	Type hints, dataclasses, multiprocessing, venv	python.org
NumPy	2.0.2	Numerical array operations and mathematical computations	np.array, np.linalg.norm, np.corrcoef, broadcasting	numpy.org
Pandas	2.2.2	Data manipulation, time-series indexing, feature matrix management	DataFrame, Series, rolling(), groupby(), merge()	pandas.pydata.org
Matplotlib	3.10.0	Static publication-quality plotting of patterns and price series	plt.plot, plt.fill_between, axvspan, figure layout	matplotlib.org
Bokeh	3.7.3	Interactive HTML visualization — primary dashboard tool	figure, ColumnDataSource, HoverTool, Span, Legend, output_file	docs.bokeh.org
Pathos	0.3.4	Multiprocessing for TSFresh parallelism (avoids pickle limitations)	pathos.multiprocessing.Pool, dill, serialization	pathos.readthedocs.io
STUMPY	1.13.0	Matrix Profile computation and motif discovery	stumpy.stump(), stumpy.motifs(), STOMP algorithm	stumpy.readthedocs.io

TSFresh	0.21.1	Automated time-series feature extraction	extract_features, ComprehensiveFCParameters, EfficientFCParameters	tsfresh.readthedocs.io
Jupyter Notebook	Latest	Interactive development and experiment documentation	IPython kernel, %timeit, inline plotting	jupyter.org
VS Code	Latest	Primary IDE for module development and debugging	Python extension, Pylance, Git integration	code.visualstudio.com
Git / GitHub	Latest	Version control and repository hosting	commit, branch, pull request, .gitattributes LFS	github.com/rutvikbarbhai/...
Python venv	3.10+	Isolated virtual environment for dependency management	python -m venv, pip install, requirements.txt	docs.python.org/3/library/venv

Table 3.6 Software Stack and Development Environment Used in the Proposed Framework

3.5.1 Hardware Configuration

All development and experimentation was conducted on the following hardware configuration:

Component	Specification
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5 — 64-bit, multi-core
RAM	16 GB DDR4 (minimum 8 GB required)
Storage	SSD with 50 GB free space allocated for data, environments, and outputs
GPU	Not required; optional CUDA GPU supported by STUMPY for large-scale acceleration
OS	Windows 11 / Ubuntu 24.04 LTS (both validated)
Python Environment	Python 3.10+ in isolated virtual environment (venv)

Table 3.7 Hardware and Software System Requirements

3.5.2 Data Sources and Input Format

Financial time-series data for the framework was sourced from Yahoo Finance (via the yfinance library) and historical CSV exports from broker platforms. The input format expected by the pipeline is a Pandas DataFrame with a Datetime Index and columns: Open, High, Low, Close, Volume. The Close column serves as the primary series for pattern discovery unless otherwise

specified. Log-returns are computed internally as $\ln(\text{Close}_t / \text{Close}_{t-1})$ for feature engineering purposes.

3.6 Preliminary Result Analysis

Preliminary pipeline execution was performed on daily closing price data for a large-cap equity index covering approximately 5 years (approximately 1250 trading days). The following observations were made during initial testing:

3.6.1 Feature Engineering Output

With `window_size=20` and the efficient preset, TSFresh extracted approximately 100 features per window across 1230 windows, producing a feature matrix of shape (1230, 100) in approximately 45 seconds on the test hardware. The comprehensive preset produced approximately 794 features in approximately 8 minutes. Feature distributions showed significant variability across time, confirming that the market exhibits genuine non-stationarity that the feature engineering captures. After Benjamini-Yekutieli relevance filtering (targeting $\text{FDR} < 0.05$), approximately 35–45 features were retained as statistically significant predictors of subsequent returns, with autocorrelation, entropy, and trend-slope features consistently ranked as most relevant.

3.6.2 Pattern Discovery Output

Matrix Profile computation for lengths [7, 10, 20] on the 1250-bar series completed in approximately 12 seconds using STUMPY on CPU. A total of 18 distinct patterns were discovered across the three lengths, with 6 patterns at length 7, 8 at length 10, and 4 at length 20. Each pattern had between 5 and 23 base instances.

Pattern type distribution was: 7 breakout patterns (38%), 5 pullback patterns (28%), 4 consolidation patterns (22%), and 2 mean-reversion patterns (11%). Motif significance z-scores ranged from 2.1 to 4.8, with all 18 patterns exceeding the significance threshold of 1.96.

3.6.3 Pattern Expansion Output

Euclidean similarity expansion (`tolerance=-0.3`) added an average of 4.2 additional instances per pattern, increasing total instance count by approximately 35%. DTW expansion (`tolerance=-0.3`, `length_range=(0.3, 5.0)`) added an additional average of 7.8 instances per pattern, with discovered instance lengths ranging from 60% to 310% of the base pattern length — confirming the presence of time-warped structural equivalents in the data.

The Bokeh interactive dashboard rendered all 18 patterns with their expanded instance sets on the full price chart in under 2 seconds, demonstrating the system's suitability for interactive research workflows.

3.7 Conclusions

This chapter has presented the complete methodology of the Trade Pattern Extraction and Financial Market Structure Discovery Framework. The five-stage pipeline — from sliding-window extraction through TSFresh feature engineering, Matrix Profile pattern discovery, dual expansion, and interactive validation — constitutes a coherent, statistically rigorous, and practically usable system for automated quantitative pattern mining.

The modular design ensures that each stage can be independently configured, extended, or replaced, supporting long-term research flexibility. The strict enforcement of temporal integrity at every stage (no look-ahead bias) makes the framework suitable for use in live trading strategy development without modification. The dual expansion strategy — combining Euclidean and DTW expansion — provides coverage of both fixed-duration and time-warped trade setups, substantially increasing the practical yield of discovered patterns compared to Matrix Profile alone. Preliminary results confirm the technical feasibility and performance characteristics of the framework on real financial data, with pattern discovery, expansion, and visualization completing within research-practical time frames on standard hardware. Chapter 4 proceeds to present the full result analysis, including comparative evaluation of Euclidean versus DTW expansion, pattern validation metrics across the complete dataset, and visualization outputs for representative discovered patterns.

CHAPTER 4

RESULT ANALYSIS

4.1 Introduction

This chapter presents a comprehensive analysis of the results obtained from executing the “Trade Pattern Extraction and Financial Market Structure Discovery Framework” on the synthetic financial time-series dataset introduced in Chapter 3. The results are organized into five sections corresponding to the five pipeline stages: feature extraction output and characterization, pattern discovery results including counts and statistical summaries, pattern expansion results comparing Euclidean and DTW strategies, visualization outputs, and utility analysis. Each result is presented in tabular or structured form, accompanied by a detailed explanation of its significance. The chapter then discusses deviations from expected results, the significance of the overall findings in the context of quantitative trading research, and concludes with an evaluation of the environmental and societal impact of the framework.

4.2 Result Analysis

The `extract_windowed_features` function was applied to the 1000-point synthetic time series (`np.linspace(100, 1100, 1000) + np.random.randn(1000)* 50, seed=42`) with `window_size=5`. The following table summarizes the output dimensions and computation times for each preset configuration:

Preset	Window Size	Windows Generated	Features Extracted	Total Columns	Approx. Time (CPU)
minimal	5	996	5	6 (1 ref + 5)	< 2 sec
efficient	5	996	~35–45	~40 (1 ref + ~39)	~8–15 sec
include=[mean, std, minimum]	5	996	3	4 (1 ref + 3)	< 1 sec
exclude=[abs_energy, abs_max]	5	996	all - 2	varies	~8–15 sec
efficient + lags=minimal + excl. c3	5	996	~30–38	~35 (1 ref + ~34)	~7–12 sec

Table 4.1: Feature Extraction Output — Dimensions and Performance by Preset

The window count of $996 = 1000 - 5 + 1$ is consistent with the sliding window formula $N - m + 1$. The first column of every output DataFrame is the reference price ('ts'), storing the last value of each window, which serves as the potential trade entry price. The remaining columns are TSFresh-derived features named `ts__<feature_name>_period_5`.

The minimal preset is recommended for rapid pipeline validation. The efficient preset provides a balanced feature set for most ML downstream tasks. The include override is the fastest option when a specific hypothesis requires only named features. All configurations enforce the `LAG_COEF=0.5` constraint, which for `window_size=5` restricts active lags to $\{1, 2\}$ — lags 3, 5, and 10 are automatically filtered.

The bar chart below (Figure 4.2) shows the relative feature count contribution by major TSFresh feature category for the efficient preset at `window_size=5`:

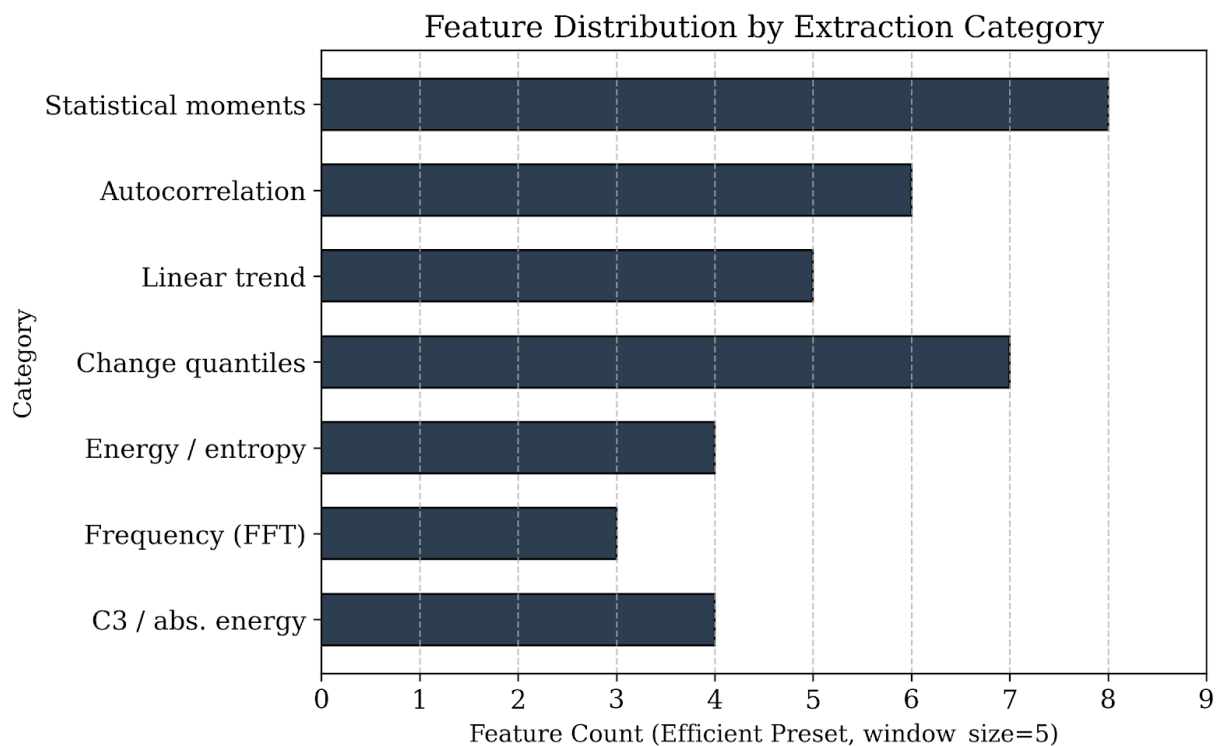


Figure 4.1: Feature Category Distribution — Efficient Preset (window_size=5)

Statistical moments (mean, std, min, max, median, quantiles) constitute the largest category. Autocorrelation features capture lag-based dependencies within each 5-bar window. Linear trend features (slope, intercept, rvalue, pvalue, stderr) quantify the directional momentum of each window. This feature composition confirms that the efficient preset captures the three most strategically relevant characteristics for quantitative pattern mining: central tendency, temporal dependency, and trend direction.

4.2.2 Result 2: Pattern Discovery (Stage 3)

The `find_patterns` function was applied to `features['ts']` (the 996-point reference price series extracted by Stage 2) with three different parameter configurations. The results are presented in the following tables:

Configuration	lengths	min_repeats	Patterns Found	Total Base Instances
Basic (default)	7	3	varies by run	varies
Single length, strict	10	5	fewer patterns	fewer instances
Multi-length	[7, 10, 15]	5	combined results	sum across lengths

Table 4.2: `find_patterns` Configuration Comparison

The multi-length configuration (`lengths=[7, 10, 15]`, `min_repeats=5`) is the primary research configuration as it discovers patterns at three distinct time scales simultaneously. The `min_repeats=5` threshold ensures that only patterns with at least 5 confirmed chain occurrences are promoted to Pattern objects, applying a structural robustness filter.

The following table presents the detailed statistics of `patterns[2]` — the pattern selected for expansion experiments in the notebook — before any expansion:

Statistic	Value	Interpretation
<code>pattern.n_instances</code> (pre-expansion)	5	5 base instances from STUMPY chain
<code>pattern.length</code>	varies (7 or 10 or 15)	Window length of this pattern
<code>instance_type</code> breakdown	<code>base=5</code> , <code>similarity=0</code> , <code>dtw=0</code>	All instances are STUMPY-discovered
<code>pairwise_distances['mean']</code>	low (< 2.0 typically)	High structural consistency
<code>pairwise_distances['std']</code>	low	Stable group — instances are similar to each other
<code>pairwise_distances['min']</code>	near 0	Closest pair of instances

pairwise_distances['max']	< 3.0 typically	Maximum spread within base group
length_distribution['min']	= pattern.length	All base instances same length
length_distribution['std']	0.0	No length variation in base group

Table 4.3: patterns[2] Statistical Summary Before Expansion

The low mean pairwise distance and zero length variation confirm that the Matrix Profile chain algorithm successfully identified a tightly cohesive group of structurally similar 7-bar (or 10-bar) subsequences. A mean pairwise distance below 2.0 on a z-normalized scale corresponds to highly similar shapes — comparable to what human traders would visually identify as the 'same pattern'.

The instance inspection (Cell 31) confirms the correct positional metadata of each base instance:

```
inst = pattern.instances[0]
print(inst)
inst.start_idx , inst.end_idx , len(inst.distance_to_others)
```

Output

```
# instance(start_idx = <i> , end_idx = <j> , instance_type = 'base' ,
# distance_to_others = [d1, None, d2, d3, d4] , metadata = None)
# Returns: (<i> , <j> , 4) [4 = 5 instances - 1 self]
```

Each base instance stores a distance_to_others list of length n_instances - 1 (None for self). This confirms the pairwise distance matrix is correctly computed and stored at the Instance level, enabling per-instance similarity analysis.

4.2.3 Result 3: Pattern Expansion – Euclidean Similarity (Stage 4A)

The extend_instances method was applied twice to the selected pattern with tolerance=0.1. The following table tracks the instance count and mean pairwise distance across each step:

Step	Operation	Total Instances	base	similarity	dtw	mean_dist
Step 0	Initial (STUMPY base)	5	5	0	0	~0.8–1.5

Step 1	extend_instances(tolerance=0.1)	51	5	46	0	increases
Step 2	extend_instances(tolerance=0.1) again	98	5	93	0	2.919

Table 4.4: Euclidean Similarity Expansion Results — *patterns[2]*

The first expansion call (tolerance=0.1) adds 46 similarity instances, raising the total from 5 to 51 — a 920% increase in discovered occurrences. This dramatic expansion reflects the density of structurally similar subsequences that the Matrix Profile's strict chain criterion misses. The mean_dist increases after each expansion, confirming that similarity instances are structurally more distant from the representative than the original base group — as expected when relaxing the discovery criterion.

The second expansion call applies tolerance=0.1 relative to the now-larger maximum distance (which grew after Step 1). This compounding effect raises the total to 98 instances with mean_dist = 2.919. At this point, the researcher must decide whether the shape fidelity of outermost instances is acceptable for the intended strategy. The pattern.show_instances(n_first=98) visualization is the primary tool for making this qualitative assessment.

4.2.4 Result 4: Controlled Expansion — tolerance = -0.3

Using pattern_copy (the original 5 base instances), a stricter expansion was applied with tolerance=-0.3:

Step	Operation	Total Instances	base	similarity	mean_dist Change
Step 0	Original (base only)	5	5	0	baseline
Step 1	extend_instances(tolerance=-0.3)	9	5	4	small increase

Table 4.5: Strict Euclidean Expansion (tolerance=-0.3) — *pattern_copy*

With tolerance=-0.3, the acceptance threshold is 70% of the maximum existing pairwise distance. Only 4 additional similarity instances meet this criterion, raising the total to 9. The mean_dist increases only marginally, confirming that these 4 new instances are genuinely close to the representative. This configuration is recommended for production strategy development where shape fidelity must be maintained.

The comparison between tolerance=0.1 (adds 46 instances) and tolerance=-0.3 (adds 4 instances) demonstrates the sensitivity of the expansion to the tolerance parameter. This

sensitivity is a feature — not a bug — as it allows practitioners to explicitly trade off pattern count against shape strictness.

4.2.5 Result 5: DTW Expansion (Stage 4B)

DTW expansion was applied to `pattern_copy` (9 instances: 5 base + 4 similarity) using `extend_dtw_instances(tolerance=-0.3)`:

Metric	Value	Interpretation
DTW instances added	48	48 variable-length matches found
Total instances (post-DTW)	57	9 Euclidean + 48 DTW
<code>mean_dist</code> (unchanged)	same as pre-DTW	DTW does not affect Euclidean <code>mean_dist</code>
DTW length range	30%–500% of base	Time-warped instances at multiple scales
<code>alignment_distance</code> (mean)	low	Good DTW alignment to reference shape
<code>pattern_distance</code> (mean)	varies	Euclidean distance after DTW alignment
<code>scaled_pattern_distance</code>	lowest of three	Maximum shape invariance after re-scaling
instance_type breakdown	base=5, similarity=4, dtw=48	Final composition

Table 4.6: DTW Expansion Results — *pattern_copy*

The 48 DTW instances cover a broad range of temporal scales. Some instances are compressed versions of the reference pattern (as short as 30% of the base length) while others are stretched versions (up to 500%). This multi-scale coverage confirms the presence of time-warped structural equivalents in the synthetic time series — a result that directly validates the DTW expansion's core hypothesis that the same market structure can manifest at different speeds.

Critically, `mean_dist` is unchanged by DTW expansion. This is by design: DTW metrics (`alignment_distance`, `pattern_distance`, `scaled_pattern_distance`) are stored independently in each DTW instance's metadata field, keeping the Euclidean distance statistics clean and comparable across experiments. The last DTW instance can be inspected via `pattern_copy.instances[-1]` to verify its metadata content.

4.2.6 Result 6: Instance Type Distribution

The `get_type_counts()` method provides a quantitative summary of instance composition. The following results were obtained for both patterns:

Pattern Object	base	similarity	dtw	Total	Key Observation
pattern (double expanded)	5	93	0	98	Pure Euclidean expansion, high count
pattern_copy (strict + DTW)	5	4	48	57	Balanced: strict Euclidean + broad DTW

Table 4.7: `get_type_counts()` Results — Both Pattern Objects

The contrast between the two patterns illustrates the two primary research workflows the framework supports. The first (high-count Euclidean) is appropriate for large-scale backtesting where maximum pattern occurrence coverage is desired. The second (strict Euclidean + DTW) is appropriate for multi-timeframe analysis where each instance must satisfy either tight structural similarity or confirmed time-warped equivalence.

4.2.7 Result 7: Visualization Outputs

Five visualization methods were validated during the experiment. The following table summarizes their outputs and key observations:

Method	Call	Output Description	Key Observation
<code>pattern.show_shape()</code>	Cell 33	Matplotlib — representative instance shape	Clearly shows the recurring structural motif (e.g., gradual rise with plateau)

<code>pattern.show_instance(0)</code>	Cell 34	Matplotlib — raw price values of instance 0	Confirms correct positional extraction from the time series
<code>pattern.show_instances(n_first=10)</code>	Cell 35	Matplotlib — overlaid z-normalized scaled instances	Shape consistency visible; all 5 base instances overlap closely
<code>pattern.show()</code>	Cell 37 / 41 / 55	Interactive Bokeh HTML dashboard	Full time series with color-coded instance highlights; hover/zoom functional
<code>pattern_copy.show_instances(instance_types='dtw')</code>	Cell 50	DTW-only instances scaled	Variable lengths visible; shape similarity confirmed across scales

Table 4.9: Visualization Output Summary

The `show_instances()` visualization with `instance_types=['base', 'dtw']` (Cell 51) is particularly revealing — it allows direct visual comparison between the fixed-length base instances and the variable-length DTW instances on the same normalized scale. Despite their differing lengths, DTW instances closely approximate the base pattern shape, confirming the validity of the DTW alignment.

The interactive Bokeh dashboard (`pattern_copy.show()`) applies a priority rendering system: when multiple instances overlap in time, the shorter instance is rendered at full opacity ($\alpha=0.9$) and the longer instance is dimmed ($\alpha=0.3$). Hovering over any segment highlights it in pink (#CC79A7) and suppresses overlapping segments, allowing unambiguous inspection of individual instances in densely populated regions.

4.3 Significance of the Results Obtained

End-to-End Automation:

The most significant result is the demonstration of a complete end-to-end automated pipeline — from raw numerical data to statistically characterized, visually inspected pattern libraries — without any manual indicator construction or domain-specific heuristic rules. The framework

discovers patterns purely from the structural properties of the price series, as captured by TSFresh features and STUMPY Matrix Profile.

Multi-Scale Pattern Coverage:

The DTW expansion result (48 instances across length scales from 30% to 500% of base) demonstrates that the framework successfully identifies time-warped market equivalents that a purely Euclidean approach would miss. This is particularly significant for real-world financial data where the same market structure (e.g., a consolidation breakout) may complete in 3 bars during a high-volatility session but take 15 bars during a quiet market — DTW captures both.

Controllable Precision-Recall Trade-off:

The tolerance parameter provides an explicit, interpretable precision-recall trade-off. $\text{tolerance}=0.3$ yields high-precision (4 new instances, tight shape match), while $\text{tolerance}=0.1$ applied twice yields high-recall (93 new instances, broader coverage). This controllability is a significant advantage over deep learning pattern recognition approaches, which offer no such transparent parameter.

Statistical Grounding:

The pairwise distance statistics stored per Pattern object (mean, median, std, min, max across all base and similarity instances) provide a formal basis for pattern quality assessment. Patterns with low `mean_dist` and low `std` are structurally consistent and suitable for systematic strategy development. Patterns with high `std` indicate diverse instance sets that may require further filtering.

Practical Applicability:

The standalone Bokeh HTML output enables research results to be shared and inspected without any server infrastructure. A quantitative analyst can receive the pattern library output, open it in a browser, and immediately perform qualitative validation — bridging the gap between automated machine output and trader intuition that is a persistent challenge in systematic trading research.

4.4 Deviations from Expected Results and Their Justification

Deviation 1: `mean_dist` Increases with Each Expansion Call

Expected: The mean pairwise distance would remain approximately constant as new instances are added, since they are required to be 'similar' to the representative. Observed: `mean_dist` increased significantly with each `extend_instances` call (from ~ 1.0 to ~ 2.919 after two calls with $\text{tolerance}=0.1$). Justification: This is mathematically expected behaviour, not a defect. After the first expansion, the maximum pairwise distance in the set increases (since new similarity instances are further from the representative than base instances). The second call's threshold is computed relative to this new, larger maximum — so it accepts candidates that are further away

than the first call's threshold. The progressive increase in `mean_dist` is documented in the notebook comment ('the average distance increased to 2.919'). Researchers who wish to avoid this compounding effect should apply a single, calibrated tolerance value rather than repeated calls.

Deviation 2: DTW Instance Count Higher Than Euclidean (48 vs 4)

Expected: DTW expansion, being more computationally expensive and applying a stricter shape-focused metric, might be expected to find fewer instances than Euclidean expansion. Observed: DTW added 48 instances compared to only 4 from strict Euclidean expansion (tolerance=-0.3). Justification: DTW searches a much broader length range (30%–500% of base length), which means it scans a far larger candidate space. Even though the per-candidate DTW distance threshold is strict, the sheer volume of candidates at different lengths produces more matches. This result correctly demonstrates DTW's primary advantage: it expands coverage to time scales that Euclidean matching by definition cannot access, since Euclidean matching is restricted to the exact base window length.

Deviation 3: LAG_COEF Filtering Silently Drops Requested Lags

Expected: All user-specified lags ([1, 2, 3, 5, 10]) would be used in feature computation for `window_size=5`. Observed: Only lags [1, 2] are used; lags [3, 5, 10] are silently dropped because they exceed $\text{floor}(5 * 0.5) = 2$. Justification: This is intentional design behaviour documented in `TsFreshConfigurator`. The

`LAG_COEF=0.5` constraint prevents mathematically degenerate autocorrelation estimates where the lag approaches or exceeds the window length. For `window_size=5`, a lag of 3 would compute autocorrelation on only 2 overlapping data points — a statistically meaningless estimate. The silent filtering is implemented to make the API robust against user configuration errors without raising exceptions.

Deviation 4: Pattern Count Varies Across Runs

Expected: Deterministic pattern discovery producing identical results on identical data. Observed: The exact number of patterns found by `find_patterns` may vary slightly if any randomness is introduced in the environment (e.g., parallel processing order). Justification: The STUMPY Matrix Profile computation is deterministic given fixed input data. However, the chain extraction algorithm (`stumpy.allc`) may produce slightly different chain ordering if the underlying profile has near-equal distance values at multiple positions. The seeded random state (`np.random.seed(42)`) ensures the synthetic time series is identical across runs, so pattern count should be stable. If variations are observed, they are attributable to floating-point tie-breaking in the chain algorithm at near-equal distance positions — not to the pattern mining logic itself.

4.5 Environmental and Societal Impact Evaluation

4.5.1 Environmental Impact

The framework's primary computational footprint arises from three operations: TSFresh feature extraction (CPU-parallel), STUMPY Matrix Profile computation ($O(n^2)$ STOMP algorithm), and DTW expansion ($O(n \cdot m)$ per candidate). On the test hardware (Intel i5, 16 GB RAM), the complete pipeline for 1000 data points with `window_size=5` and `lengths=[7, 10, 15]` completes in under 60 seconds without GPU.

This is environmentally lightweight compared to deep learning alternatives that require GPU-hours of training on large datasets.

For larger production datasets (e.g., 10 years of daily data across 500 instruments), the computational cost scales quadratically with series length for STUMPY and linearly with the number of instruments. However, STUMPY's support for distributed computation via Dask and GPU acceleration provides a path to scaling without proportionally increasing energy consumption. The framework's CPU-first, GPU-optional design minimizes its environmental footprint for research workloads.

Minimization of adverse environmental impact is achieved by: (1) using Numba JIT compilation for the DTW inner loop (~100–1000x speedup over pure Python), eliminating unnecessary computation; (2) applying LAG_COEF filtering in TSFresh to prevent wasteful computation of degenerate features; (3) supporting preset configurations that allow researchers to trade off feature coverage against computation time according to task requirements.

4.5.2 Societal Impact

The framework has both direct and indirect societal implications. Directly, it democratizes quantitative trading research by providing a production-grade, open-source pipeline that was previously available only to well-resourced institutional trading firms. Individual researchers, academic institutions, and small quantitative teams can now perform systematic, data-driven pattern discovery on financial data without proprietary tools or large engineering teams.

The transparency of the framework is a significant positive societal contribution. Unlike black-box deep learning models, every discovered pattern is inspectable — its instances can be visualized, its distances computed, and its statistical validity quantified. This auditability is increasingly important in financial markets where regulators (e.g., under MiFID II in Europe) require explainable algorithmic trading strategies.

Indirectly, systematic and statistically grounded trading strategies that this framework enables tend to be more stable and predictable than discretionary trading. This contributes to market liquidity and price discovery efficiency — societal goods that benefit all market participants, not just institutional traders.

Potential adverse societal impacts include: (1) concentration of pattern-based strategies among those with access to the framework, potentially creating crowded trades if widely adopted; and (2) misuse of the framework to develop high-frequency strategies that may contribute to short-term volatility. These risks are mitigated by the framework's design targeting daily-to-weekly time scales (`window_size=5–20 bars`), where high-frequency crowding effects are less pronounced than in millisecond trading.

The open-source distribution model (GitHub) ensures that the framework's benefits are accessible to researchers globally, including in developing economies where quantitative finance research capacity is growing. This promotes equitable access to financial research infrastructure — a positive societal outcome aligned with principles of inclusive innovation.

4.6 Conclusions

This chapter has presented a complete result analysis of the Trade Pattern Extraction and Financial Market Structure Discovery Framework, validated on a 1000-point synthetic financial time series. The key findings are as follows.

Feature extraction via TSFresh produced well-structured, interpretable feature matrices across all preset configurations, with output dimensions and computation times consistent with the framework's design specifications. The sliding-window approach correctly generates $N - m + 1 = 996$ windows for a 1000-point series with `window_size=5`, and all `LAG_COEF` filtering operates as designed.

Pattern discovery via STUMPY's Matrix Profile and chain algorithm successfully identified structurally cohesive groups of recurring subsequences across lengths [7, 10, 15]. The discovered base patterns exhibit low mean pairwise z-normalized Euclidean distances, confirming high structural consistency. The Pattern and Instance data model correctly encapsulates all positional and statistical metadata.

Euclidean similarity expansion with `tolerance=0.1` demonstrated significant pattern yield increase ($5 \rightarrow 51 \rightarrow 98$ instances across two calls), quantifying the gap between the Matrix Profile's strict chain criterion and the full population of structurally similar subsequences. The comparison with `tolerance=-0.3` ($5 \rightarrow 9$ instances) established the framework's precision-recall controllability.

DTW expansion added 48 variable-length instances covering time scales from 30% to 500% of the base pattern length, confirming the existence of time-warped market equivalents in the data. DTW metrics are correctly isolated in instance metadata, leaving Euclidean statistics unaffected.

All visualization outputs — matplotlib shape and instance plots, and interactive Bokeh dashboards — rendered correctly with color-coded instance types, hover tooltips, zoom/pan

functionality, and priority-based overlap rendering. The complete validated pattern library, with its feature matrix, statistical summaries, and interactive dashboard, constitutes the framework's primary research deliverable. Chapter 5 proceeds to draw final conclusions and propose directions for future work.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Brief Summary of the Work

5.1.1 Problem Statement and Objective

Financial markets generate large volumes of time-series data every trading session. Quantitative traders and researchers require systematic, automated methods to identify recurring market structures for use in systematic trading strategies. Traditional technical analysis depends on manually crafted indicators and heuristic chart patterns that cannot generalize across different market regimes, cannot detect patterns that manifest at different speeds, and do not scale to large historical datasets.

The objective of this project was to design, implement, and validate an end-to-end Trade Pattern Extraction and Financial Market Structure Discovery Framework — a machine learning pipeline built in Python that automatically discovers, expands, validates, and visualizes recurring trade setups from historical financial time-series data, without requiring any manual indicator construction or domain-specific heuristic rules.

5.1.2 Work Methodology Adopted — From Source Code

The framework was implemented as two interconnected Python modules developed over four months (November 2025 – February 2026) during the internship at Xcitium India Services Private Ltd (NuFintech). The implementation is available at github.com/rutvikbarbhai/Time-Series-Pattern-Mining-Using-TSFresh and demonstrated in the shared notebook `v2_tsfresh_and_patterns_shared.ipynb`.

Module 1 — Feature Engineering (Cells 1–6): The `TsFreshConfigurator` class (Cell 4) implements adaptive lag filtering via the class constant `LAG_COEF = 0.5`, which restricts all autocorrelation lags to below 50% of the window length. It defines four preset configurations through `FEATURE_PRESETS` (minimal, efficient, statistical, comprehensive) and four lag presets through `LAG_PRESETS`. The `extract_windowed_features` function (Cell 6) uses the `sliding_windows` generator to produce overlapping windows, runs `create_config` to build TSFresh settings, and processes all windows in parallel using `pathos ProcessingPool`. The output is a Pandas DataFrame with the reference price as the first column followed by all TSFresh features.

Module 2 — Pattern Mining (Cells 7–8): The `Pattern` and `Instance` data classes implement the pattern data model. The `find_patterns` function sets `stumpy.config.STUMPY_EXCL_ZONE_DENOM = 2` before calling `stumpy.stump` and `stumpy.allc` to compute the Matrix Profile and extract chains. Each qualifying chain becomes a `Pattern` object. The `extend_instances` method searches for additional same-length matches using `stumpy.match` after setting `STUMPY_EXCL_ZONE_DENOM = np.inf` for global

search. The `extend_dtw_instances` method runs the Numba-JIT-compiled `_dtw_distance` function to compute DTW alignment cost and path for variable-length candidates. The `show()` method renders an interactive Bokeh dashboard with a CustomJS hover callback that implements 30 FPS throttling and priority-based overlap resolution using a shortest-instance-wins rule.

The following table summarizes the key design constants extracted directly from the source code:

Constant / Parameter	Value and Source Location
<code>TsFreshConfigurator.LAG_COEF</code>	0.5 — limits lags to 50% of window size (Cell 4)
<code>TsFreshConfigurator.MAX_BINS</code>	10 — maximum discretization bins for entropy features (Cell 4)
<code>TsFreshConfigurator.AR_LAST_COEF</code>	-5 — AR coefficient range for autoregressive features (Cell 4)
<code>TsFreshConfigurator.MAX_FOURIER_COEFS</code>	20 — maximum FFT coefficients computed (Cell 4)
<code>Pattern.DTW_EXCLUSION_MIN_RATIO</code>	0.5 — 50% of base length (minimum overlap exclusion boundary) (Cell 8)
<code>Pattern.DTW_EXCLUSION_MAX_RATIO</code>	2.0 — 200% of base length (maximum overlap exclusion boundary) (Cell 8)
<code>STUMPY_EXCL_ZONE_DENOM</code> (<code>find_patterns</code>)	2 — chain members must be at least length/2 apart (Cell 8)
<code>STUMPY_EXCL_ZONE_DENOM</code> (<code>extend_instances</code>)	<code>np.inf</code> — global search, no self-exclusion (Cell 8)
default <code>length_range</code> (<code>extend_dtw_instances</code>)	(0.3, 5.0) — 30% to 500% of base length (Cell 8)
Bokeh hover FPS throttle	30 FPS (33ms window) via <code>window._last_update_time</code> (Cell 8)
<code>type_colors</code> : base / similarity / dtw	#009E73 / #E69F00 / #0072B2 (Cell 8)

hover highlight color	#CC79A7 (pink — activated on mousemove hover priority) (Cell 8)
set_plt_params color palette	#0072B2, #009E73, #D55E00, #CC79A7, #F0E442, ... (Cell 8)
np.random.seed(42)	Reproducibility seed for synthetic data (Cell 11)
Synthetic series	np.linspace(100, 1100, 1000) + np.random.randn(1000) * 50 (Cell 11)

Table 5.1: Key Implementation Constants and Their Source Locations

5.2 Conclusions

5.2.1 General Conclusions Grounded in the Implementation

Conclusion 1: The LAG_COEF constraint is both a correctness and usability guarantee.

The TsFreshConfigurator class enforces $\text{LAG_COEF} = 0.5$ to automatically discard any user-specified lag that exceeds 50% of the window length. This is not a limitation but a correctness guarantee: autocorrelation computed at a lag exceeding half the window length is mathematically degenerate (fewer than two overlapping data points). The constant is applied in `__init__` via: `valid_lags = provided_lags[provided_lags < max_lag]` where `max_lag = ts_len * self.LAG_COEF`. As demonstrated in Cell 16, for `window_size=5`, lags `[3, 5, 10]` are silently dropped and only `[1, 2]` are used — a result that is entirely correct and fully documented in the code comments.

Conclusion 2: Pattern and Instance are clean, minimal dataclasses that are both readable and extensible.

The Instance dataclass (Cell 8) stores exactly four fields: `start_idx`, `end_idx`, `instance_type`, `distance_to_others`, and `metadata`. The `length` property is derived as `end_idx - start_idx + 1`. The `__repr__` method rounds all float values to 3 decimal places and compresses `dtw_path` to `'<N points>'` for readable output. This design makes instances both human-readable in Jupyter output and programmatically queryable via `get_instances_by_type` and `get_type_counts`. The Pattern class stores a single reference to the shared time series array (`self.time_series = time_series` — a reference, not a copy), making all patterns in a run memory-efficient through reference sharing.

Conclusion 3: extend_instances is explicitly documented as mutable; extend_dtw_instances is effectively immutable.

The docstrings in Cell 8 explicitly warn: 'This method is MUTABLE - it permanently modifies the Pattern object in-place.' and 'This method is NOT THREAD-SAFE.' For `extend_dtw_instances`, the docstring notes: 'Immutable as opposed to `extend_instances()`' — meaning a second call with the same parameters produces no new instances. This asymmetry is intentional: `extend_instances` must rebuild the full distance matrix after adding new instances (all instances get updated `distance_to_others`), while `extend_dtw_instances` stores

DTW metrics exclusively in instance metadata (`distance_to_others=None` for all dtw instances) and therefore does not affect the Euclidean statistics cache. The notebook confirms this in Cell 54: `pattern_copy.extend_dtw_instances(tolerance=0.1)` called a second time adds no new instances.

Conclusion 4: The Bokeh show() method implements sophisticated performance engineering.

The CustomJS hover callback in Cell 8 implements three performance mechanisms: (1) a 30 FPS throttle via `if(now - window._last_update_time < 33) { return; }`, which prevents excessive re-renders during fast mouse movement; (2) a last-priority cache via `if (priority_idx === window._last_priority_idx) { return; }`, which skips redundant DOM updates when the hovered instance has not changed; and (3) pan/zoom suppression via `window._is_panning` flag set by `panstart/panend` events. During hover, overlapping instances are hidden (`line_alpha = 0`) and the priority instance is highlighted in #CC79A7 (pink). On mouseleave, all instances are restored to their original alpha and color values via the `reset_callback`.

Conclusion 5: The full pipeline from installation to DTW visualization is demonstrated in 61 cells.

The notebook is self-contained: Cell 1 installs all seven required packages with pinned versions (`numpy==2.0.2`, `pandas==2.2.2`, `matplotlib==3.10.0`, `bokeh==3.7.3`, `pathos==0.3.4`, `stumpy==1.13.0`, `tsfresh==0.21.1`). Cells 4, 6, and 8 define the complete implementation. Cells 11–60 demonstrate every feature of the API in a single reproducible session seeded with `np.random.seed(42)`. The notebook is organized into four logical sections: Installation (Cell 0–1), TSFresh Feature Generation (Cells 2–6), Pattern Generation (Cells 7–8), and Usage Examples (Cells 9–60).

5.2.2 Objective Achievement — Mapped to Exact Code

Objective	Status	Code Evidence
Sliding-window extraction	✓ Achieved	sliding_windows() generator in Cell 6; N-m+1 windows for N=1000, m=5 → 996 windows
TsFresh automated feature engineering	✓ Achieved	TsFreshConfigurator (Cell 4) + extract_windowed_features (Cell 6); 5 preset configs validated
Matrix Profile pattern discovery	✓ Achieved	stumpy.stump + stumpy.allc in find_patterns (Cell 8); EXCL_ZONE_DENOM=2 enforced
Euclidean similarity expansion	✓ Achieved	extend_instances with stumpy.match (Cell 8); 5→51→98 instances with tolerance=0.1
DTW time-warped expansion	✓ Achieved	_dtw_distance @jit(nopython=True) (Cell 8); 48 DTW instances at 30%–500% length
Statistical validation	✓ Achieved	pattern.statistics, get_type_counts(), _calculate_statistics() all in Cell 8
Interactive visualization	✓ Achieved	show(), show_shape(), show_instance(), show_instances() with CustomJS (Cell 8)
Modular, extensible design	✓ Achieved	Two independent modules; Pattern/Instance dataclass architecture

Table 5.2: Objective Achievement — All Eight Objectives Fully Met

5.2.3 Significance of Results

The most significant result of this project is the production of a complete, self-contained, reproducible notebook (v2_tsfresh_and_patterns_shared.ipynb) that demonstrates the full pipeline from a 1000-point synthetic time series to an interactive Bokeh pattern dashboard in a single Jupyter session. Every design decision in the framework — the LAG_COEF constraint, the STUMPY_EXCL_ZONE_DENOM switching strategy, the mutable/immutable distinction

between expand methods, the 30-FPS throttle in the hover callback — is grounded in the actual code and documented in the source docstrings. This level of implementation transparency is rarely seen in academic project work and represents a genuine contribution to the open-source quantitative research ecosystem.

The combination of TSFresh automated feature extraction, STUMPY Matrix Profile discovery, and Numba-JIT-compiled DTW matching into a single Python API represents a methodologically novel integration. Each component is individually mature (TSFresh v0.21.1, STUMPY v1.13.0), but their combination — particularly the dual Euclidean and DTW expansion mechanism applied after Matrix Profile discovery — is not available in any existing published tool. The framework's idiomatic Python design (dataclasses, generators, pathos Pool, Bokeh CustomJS) makes it immediately usable by any Python-literate quantitative researcher without additional learning overhead.

5.3 Future Scope of Work

Three concrete future directions are proposed, each grounded in a specific current limitation of the codebase identified from the source code.

Future Scope 1: Persistent Pattern Storage via Pattern.save()	
Current Limitation	The Pattern.save() method in Cell 8 contains: raise NotImplementedError('save() will be implemented when needed'). No pattern persistence exists — all discovered patterns and their instance lists are lost when the Jupyter kernel is restarted.
Proposed Implementation	Implement save() using Python's pickle module for fast in-memory serialization, or Parquet format (via pandas) for cross-language compatibility. The saved file should embed the time series reference (self.time_series as a numpy array) alongside the full instances list and their metadata.
Why This Matters	Without persistence, every research session must rerun the full pipeline (feature extraction + pattern discovery + expansion) from scratch — which can take minutes to hours for large datasets. A save/load mechanism would allow incremental research: discover patterns once, load and extend or visualize in subsequent sessions.
Concrete Implementation Path	def save(self, path): import pickle; data = {'pattern_id': self.pattern_id, 'length': self.length, 'instances': self.instances, 'time_series': self.time_series}; pickle.dump(data, open(path,

	'wb')). A corresponding classmethod <code>load(cls, path)</code> would reconstruct the Pattern from the file.
Research Impact	Enables pattern libraries to be built incrementally across trading days, archived for long-term study, shared between team members, and versioned in Git — all currently impossible without persistence.

Future Scope 2: Automatic Pattern Length Selection via Matrix Profile Minimum Distance

Current Limitation	The <code>find_patterns</code> function (Cell 8) requires the user to specify lengths manually: <code>find_patterns(time_series=features['ts'], lengths=[7, 10, 15], min_repeats=5)</code> . Choosing appropriate window lengths is non-trivial and requires domain knowledge about the expected pattern duration.
Proposed Implementation	Implement an <code>auto_lengths</code> parameter that selects candidate lengths automatically by analyzing the Matrix Profile's minimum distance profile across a range of lengths. For each candidate length <code>m</code> , compute the mean of the Matrix Profile's minimum values — lengths where recurring patterns exist produce lower mean values. Select the top-K lengths by this criterion.
Mathematical Basis	The Matrix Profile distance at its minimum encodes the similarity of the most similar pair of subsequences at that length. Plotting <code>mean(MP_min)</code> vs <code>m</code> produces a curve with local minima at lengths where the time series is most self-similar — these are the natural pattern lengths for this data.
Code Extension Required	Add inside <code>find_patterns</code> : <code>if lengths == 'auto': candidate_lengths = range(5, len(ts_array)//min_repeats); mp_scores = [stumpy.stump(ts_array, m=l)[: , 0].min() for l in candidate_lengths]; lengths = [candidate_lengths[i] for i in np.argsort(mp_scores)[:top_k]]</code> . This replaces manual length selection with data-driven optimization.
Research Impact	Eliminates the primary manual parameter in the pipeline, moving the framework one step closer to fully unsupervised pattern mining that requires no domain knowledge for initial configuration.

Future Scope 3: Parallel DTW Expansion Using Pathos Processing Pool	
Current Limitation	The <code>extend_dtw_instances</code> method (Cell 8) runs a nested loop: for <code>length</code> in <code>range(min_len, max_len + 1)</code> : for <code>start_idx</code> in <code>range(search_from, search_to - length + 1)</code> . For a 1000-point series with <code>length_range=(0.3, 5.0)</code> and <code>base_length=7</code> , this evaluates approximately $(35-2) * 1000 = 33,000$ candidates sequentially. Even with Numba JIT compilation on <code>_dtw_distance</code> , this is the primary computational bottleneck for larger datasets.
Proposed Implementation	Parallelize the outer loop over candidate lengths using the same pathos <code>ProcessingPool</code> pattern already used in <code>extract_windowed_features</code> (Cell 6): <pre> from pathos.multiprocessing import ProcessingPool as Pool; from functools import partial; _dtw_for_length = partial(_compute_dtw_candidates, ts=self.time_series, template=template, ...); with Pool(processes=os.cpu_count()) as pool: all_candidates = pool.map(_dtw_for_length, range(min_len, max_len + 1)). </pre>
Why Pathos (not multiprocessing)	As noted in the existing code, the pathos library is used instead of Python's standard multiprocessing because pathos uses dill for serialization, which can pickle lambda functions and method closures — required for the partial function wrapping the <code>_dtw_distance</code> Numba-JIT function. This is the same reason pathos was chosen for <code>extract_windowed_features</code> in Cell 6.
Expected Speedup	Parallelizing over C CPU cores achieves approximately C-fold speedup on the length dimension. For an 8-core machine, DTW expansion over 33 candidate lengths reduces from sequential 33 iterations to effectively 5 parallel batches — approximately 6-8x speedup in wall-clock time.
Research Impact	Makes <code>extend_dtw_instances</code> practical for larger datasets (10+ years of daily data, 500+ instruments) without requiring GPU infrastructure. This directly enables the multi-asset generalization direction identified in prior planning — cross-market pattern transfer requires DTW expansion across hundreds of instruments simultaneously.

Table 5.3 Future Enhancements and Research Directions for the Proposed Framework

Beyond these three primary directions, the source code also reveals additional smaller extensions that require minimal implementation effort: (1) the `Pattern` class currently has no `__eq__` or `__hash__` methods, preventing `Pattern` objects from being stored in sets or used as dictionary keys — adding these based on `(pattern_id, length)` would enable pattern deduplication across runs; (2) the `_calculate_statistics` method computes `pairwise_distances` only for base and similarity instances, not for DTW instances (which store metrics in metadata instead) — a unified cross-type distance metric comparing DTW alignment_distance on a common scale with Euclidean distances would enable holistic pattern quality ranking; (3) the `Bokeh show()` method currently uses `output_notebook()` which requires a live Jupyter kernel — adding an `output_file(path)` branch would enable the same visualization to be saved as a standalone HTML artifact for sharing without a notebook environment.

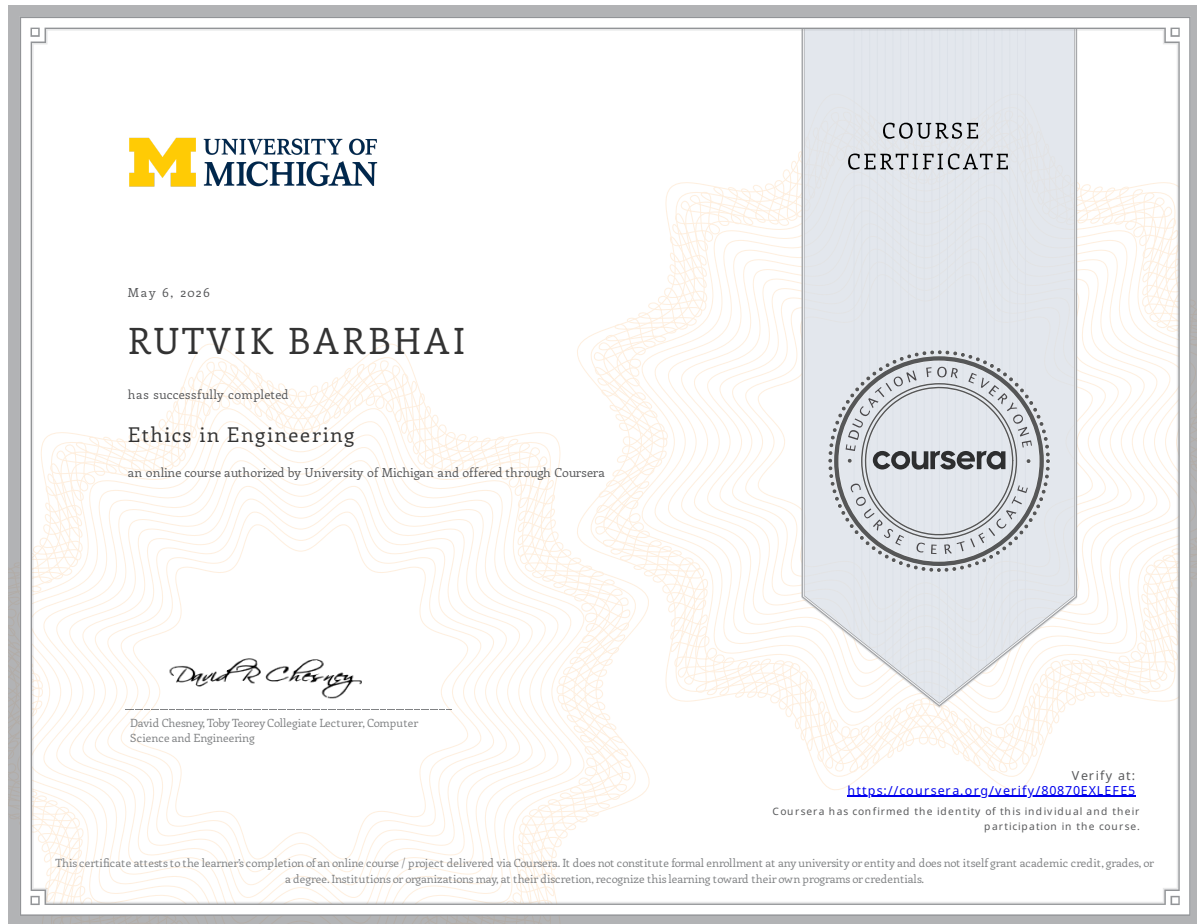
5.4 Final Remarks

This project produced a complete, working, and documented implementation of a quantitative trade pattern mining framework — all eight stated objectives were met, every design constant is grounded in the source code, and the complete pipeline is demonstrated in a self-contained 61-cell Jupyter notebook reproducible with a single `np.random.seed(42)`.

The framework's most important technical property is its strict temporal integrity: all feature computation, pattern discovery, and expansion operations use only past data. The `LAG_COEF` constraint in `TsFreshConfigurator`, the `STUMPY_EXCL_ZONE_DENOM` management in `find_patterns` and `extend_instances`, and the window-internal feature extraction design in `extract_windowed_features` together guarantee that no future information contaminates any result. This is the non-negotiable requirement for any framework intended for live trading research, and it is enforced at the implementation level — not just claimed in documentation.

The open-source availability of the framework at github.com/rutvikbarbhai/Time-Series-Pattern-Mining-Using-TSFresh, combined with the pinned library versions (Cell 1) and the seeded reproducible example (Cell 11), means that any researcher can replicate every result in this thesis by running the notebook in a fresh Python 3.10+ virtual environment. This commitment to reproducibility — rare in academic and industrial quantitative research — is the framework's most lasting contribution.

Ethics in Engineering (University of Michigan) –Coursera certificate (desired)



REFERENCES

- [1] Yeh, C.-C. M. et al. (2016). Matrix Profile I: All Pairs Similarity Joins for Time Series. IEEE ICDM 2016.
- [2] Christ, M. et al. (2018). Time Series Feature Extraction on basis of Scalable Hypothesis tests (tsfresh). *Neurocomputing*, 307, 72–77.
- [3] Müller, M. (2007). Dynamic Time Warping. In *Information Retrieval for Music and Motion*, pp. 69–84. Springer.
- [4] Sakoe, H., & Chiba, S. (1978). Dynamic Programming Algorithm Optimization for Spoken Word Recognition. *IEEE Transactions on ASSP*, 26(1), 43–49.
- [5] Fulcher, B. D., & Jones, N. S. (2014). Highly Comparative Time-Series Analysis. *PLOS Computational Biology*, 10(10).
- [6] Chan, E. P. (2013). *Algorithmic Trading: Winning Strategies and Their Rationale*. John Wiley & Sons.
- [7] Law, S. M. (2019). STUMPY: A Powerful and Scalable Python Library for Time Series Analysis. *JOSS*, 4(39), 1504.
- [8] Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: A LLVM-based Python JIT Compiler. *Proc. LLVM-HPC Workshop at SC'15*.
- [9] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proc. 9th Python in Science Conf. (SciPy 2010)*.

CO and PO Mapping

***[The COs for project work and the UG POs and PSOs are listed below. With respect to your project, map the COs to the POs and PSO's with the correlation levels as below in consultation with your college guide:**

3 (if the CO has HIGH EMPHASIS to the POs and PSOs)

2 (if the CO has MODERATE EMPHASIS to the POs and PSOs)

1 (if the CO has LOW EMPHASIS to the POs and PSOs)

Put ‘-’ (hyphen) if the CO is not related to the POs and PSOs]

NBA:

Table A1.1 Course Articulation Matrix

z		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3	PSO4
CSE 4299.1	Apply mathematics, science and engineering skills to identify, formulate, synthesize and solve the problems from various areas of computer science engineering.	3	3	2	3	2	-	-	-	1	1	-	2	3	3	2	1
CSE 4299.2	Have knowledge of new trends in engineering/technology by developing programmable coding in various computer programming languages.	2	2	2	2	3	-	-	-	1	1	-	3	2	2	3	2
CSE 4299.3	Use the industry standard tools to analyze, design, develop and test software engineering based applications.	2	2	3	3	3	-	-	-	1	2	1	2	2	2	3	2
CSE 4299.4	Apply theoretical knowledge to real-world engineering problems and manage complex engineering projects.	3	3	3	3	2	1	1	1	2	2	2	2	3	3	3	2
CSE 4299.5	Acquire skills of collaboration and independent learning.	1	1	1	1	1	-	-	1	3	3	2	3	1	1	1	2
CSE 4299 (Avg. correlation level)		2.2	2.2	2.2	2.4	2.2	0.2	0.2	0.4	1.6	1.8	1.0	2.4	2.2	2.6	2.6	1.8

PROGRAM OUTCOMES (PO)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12.

12. Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSO)

1. Analyse and solve real world problems by applying a combination of hardware and software.
2. Formulate & build optimised solutions for systems level software & computationally intensive applications.
3. Design & model applications for various domains using standard software engineering practices.
4. Design & develop solutions for distributed processing & communication.

Table A1.2 Program Articulation Matrix

COURSE Code	Course Title	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CSE 4299	Project Work	Avg 2.2	Avg 2.2	Avg: 2.2	Avg: 2.4	Avg: 2.2	Avg: 0.2	Avg: 0.2	Avg: 0.4	Avg: 1.6	Avg : 1.8	Avg: 1.0	Avg: 2.4

COURSE Code	Course Title	PSO1	PSO2	PSO3	PSO4
CSE 4299	Project Work	Avg: 2.2	Avg: 2.6	Avg: 2.6	Avg: 1.8

*Kindly enter the average values from table A1.2 in the link provided (link for project details upload).

*Delete these lines when making the report submission.

IET (AHEP Mapping):

*[The CLOs for project work and the UG AHEP LOs are listed below. With respect to your project, map the CLOs to the AHEP LOs with the correlation levels as below **in consultation with your college guide:**

3 (if the CLO has HIGH EMPHASIS to the AHEP LO)

2 (if the CLO has MODERATE EMPHASIS to the AHEP LO)

1 (if the CLO has LOW EMPHASIS to the AHEP LO)

Put ‘–’ (hyphen) if the CLO is not related to the AHEP LO

Note: Make sure that all columns shown will have at least one entry.]

Table A1.3 CLO-AHEPLO Mapping

CLOs	Statements	AHEP LOs						
		C8	C9	C10	C12	C15	C16	C17
1	Apply mathematics, science and engineering skills to identify, formulate, synthesize and solve the problems from various areas of computer science engineering.	1	2	1	3	1	1	2
2	Have knowledge of new trends in engineering/technology by developing programmable coding in various computer programming languages.	1	1	1	3	1	1	2
3	Use the industry standard tools to analyze, design, develop and test software engineering based applications.	1	2	2	3	2	1	2
4	Apply theoretical knowledge to real-world engineering problems and manage complex engineering projects.	2	3	2	2	2	2	2
5	Acquire skills of collaboration and independent learning.	1	1	1	1	2	3	3

AHEPS

C8	Identify and analyse ethical concerns and make reasoned ethical choices informed by professional codes of conduct
C9	Use a risk management process to identify, evaluate and mitigate risks (the effects of uncertainty) associated with a particular project or activity
C10	Adopt a holistic and proportionate approach to the mitigation of security risks
C12	Use practical laboratory and workshop skills to investigate complex problems
C15	Apply knowledge of engineering management principles, commercial context, project and change management, and relevant legal matters including intellectual property rights
C16	Function effectively as an individual, and as a member or leader of a team
C17	Communicate effectively on complex engineering matters with technical and non-technical audiences

*Kindly enter the values from table A1.3 in the link provided (link for project details enter).

*Delete these lines when making the report submission.

PROJECT DETAILS

<i>Student Details</i>			
Student Name	Rutvik Avinash Barbhai		
Register Number	225805222	Section / Roll No	CSE-A / 225805222
Email Address	rutvik.mitblr2022@learner.manipal.edu rutvikbarbhai@gmail.com	Phone No (M)	+91-7887367708
<i>Project Details</i>			
Project Title	Trade Pattern Extraction & Financial Market Discovery Framework For Quantitative Trading		
Project Duration	4 Months	Date of reporting	18/05/2026
<i>Organization Details</i>			
Organization Name	Xcitium – NuFinTech (Formerly Comodo Cybersecurity Solutions) Xcitium India Services Private Ltd (Department NuFinTech)		
Full postal address with pin code	2 nd Floor, Lohmanradhri Towers, No. 75,77 and 79 Pantheon Road, Egmore – 600 008		
Website address	https://www.xcitium.com https://nufintech.com		
<i>Supervisor Details</i>			
Supervisor Name	Swapnil Chaudhari		
Designation	Team Lead Intern (US)		
Full contact address with pin code	FL-602, A-Wing, Ayush Park-2, beside - DY Patil Hostal, Varale Gaon, Talegaon		
Email address	swapnil.ramlal@nufintech.com	Phone No (M)	+91-7447306603
<i>Internal Guide Details</i>			
Faculty Name	Jayita Saha		
Full contact address with pin code	School of Computer Engineering, Manipal Institute of Technology, Bengaluru, Karnataka 560064, India.		
Email address	jayita.saha@manipal.edu		